CSC 648 / CSC 848 — Software Engineering
Section 03 | Spring 2026

Milestone 5 — Final Delivery (M5)

Home4U

AI-Assisted Interior Design Recommendation Platform

Team Number	4
Team Alias	Vibecoding for Internship
Date	May 20, 2026
Deployment URL	https://home4uu.duckdns.org/

Team Members

Name	Role
Caleb Ponce (cponce8@sfsu.edu)	Team Lead, Backend/Auth, Deployment
Mason Lee	Frontend/UI
Christopher Quach	Backend/API & Database Alignment
Tyler Morris	QA / Smoke Tests / Deployment Verification
Dias Almat	Data (Styles/Tags/Search) & Recommendations

Revision History

Version	Date	Author	Summary of Changes
M5	May 20, 2026	Team 4	Final M5 submission. Cover, TOC, Product Summary, Team Contributions, Lesson
M4v2	May 20, 2026	Team 4	Updated deployment URL to https://home4uu.duckdns.org/ (HTTPS secured). No c
M4v1	May 12, 2026	Team 4	Participant table updated to 15 users. Unique features table updated to match profe
M4v1	April 28, 2026	Team 4	Initial M4v1 draft: product summary, usability test plan, QA test plan, AI ethics revie
M3v2	April 29, 2026	Team	Implemented Figma designs to showcase UI/UX correctly.
M3v2	April 21, 2026	Team	Redid diagrams to match frontend designs. Reworked wireframes.
M3v1	April 14, 2026	Dias, Team	Added diagrams, design patterns.
M2v2	March 5, 2026	Caleb Ponce	Revised M2: updated team contributions/scores, ensured all requirements met.
M2v1	March 4, 2026	Team	Final Milestone 2 submission.

M2v0.9	March 3, 2026	Team	Completed architecture diagrams, APIs, and deployment documentation.
M2v0.7	March 2, 2026	Team	Drafted data definitions and functional requirements.
M2v0.5	March 1, 2026	Team	Created Milestone 2 document structure.
M1v2	Feb 19, 2026	Team	Revised Milestone 1 based on instructor feedback.
M1v1	Feb 8, 2026	Team	Initial Milestone 1 submission.
M1v0.1	Feb 1, 2026	Team	Initial project draft.

Table of Contents

1. Product Summary

1.1 Product Name

Home4U — AI-Assisted Interior Design Recommendation Platform

1.2 Summary of Major Features and Services

Home4U is a full-stack web application that helps users visualize, plan, and shop for interior design transformations. At its core, the platform accepts a room photo upload and uses a deterministic image analysis pipeline — extracting signals such as brightness, saturation, dominant color, warmth bias, and aspect ratio — to score the room against twelve or more curated design styles including Modern, Scandinavian, Industrial, Bohemian, Japandi, Mediterranean, Farmhouse, Coastal, and others. The resulting style scores are normalized to spread meaningfully across a wide range so users can clearly identify their best-fit style.

Once a style is selected and a plan is generated, Home4U produces a budget-tiered shopping recommendation list organized into categories such as Furniture, Lighting, Textiles, and Decor. Each item is linked to real retailer search URLs — IKEA, Wayfair, Target, and Amazon — pre-filtered to the user's chosen budget tier (low, medium, or high). The before/after visualization panel applies style-specific CSS filter stacks to the uploaded room image, giving users a genuine visual preview of how the chosen style transforms the space.

The platform includes a Style Explorer with palettes, material signatures, and mood descriptors for each design style, a saved Projects dashboard where users can name, revisit, and track multiple room plans, and full user authentication with JWT tokens and bcrypt password hashing. The production system is deployed at https://home4uu.duckdns.org/ with HTTPS secured via Let's Encrypt, an nginx reverse proxy, a FastAPI/Uvicorn backend, and a React/Vite frontend.

1.3 Unique or Distinguishing Features

- Style-specific CSS filter stacks on the After panel — each design style applies a distinct photographic transformation (industrial: desaturation + sepia; coastal: blue hue-rotate; bohemian: saturation boost; Japandi: warm sepia + softened contrast) creating visually meaningful before/after previews without a paid image-generation API.
- Score normalization engine that spreads clustered style match scores across a wider meaningful range, preserving relative order while ensuring the best-fit style is clearly distinguishable from weaker matches.
- Budget-tiered shopping plan (low/medium/high) that auto-generates retailer search links pre-filtered to the user's price range using retailer-specific URL price parameters for all four supported stores.
- Room state signal extraction (openness, clutter level, contrast, furnishing density) derived entirely from image profiles and tag inference — no external AI API required.
- Production HTTPS deployment with automatic DuckDNS IP synchronization (systemd timer, every 5 minutes) and a PM2-managed process stack for reliable process management.

1.4 Deployment URL

https://home4uu.duckdns.org/

2. Merged Milestone Documentation

The following milestone documents are appended in full immediately after this section, in the order listed below. Internal tables of contents and section numbering within each document have not been reorganized.

- 2.1 Milestone 1 — Version 2 (M1v2)
- 2.2 Milestone 2 — Version 2 (M2V2)
- 2.3 Milestone 3 — Version 2 (M3V2)
- 2.4 Milestone 4 — Version 2 (M4v2)

3. Team Contributions

The following table summarizes each member's contributions across the full semester, as assessed by the Team Lead.

Team Member	Role(s)	Contribution Summary	Score (1–10)
Caleb Ponce (Team Lead) cponce8@sfsu.edu	Team Lead Backend/Auth Deployment	Led all milestone coordination and repository management from M1 through M5. Architected	10
Mason Lee	Frontend/UI	Designed and implemented the core frontend pages including Dashboard, Workspace, and	9
Christopher Quach	Backend/API Database	Implemented backend API routes, database schema alignment, and ORM models. Contribu	8
Tyler Morris	QA / Smoke Tests	Wrote and maintained smoke test suites for API, auth, search, and workspace flows. Led QA	9
Dias Almat	Data Docs	Built the style, tag, and search data layer and recommendation seeding. Contributed to mile	9

Team Lead Statement

Each member of Team 4 contributed meaningfully across the full semester. Caleb led project architecture and deployment from the first milestone through final delivery. Mason drove the entire frontend visual experience and user-facing quality. Christopher ensured the backend API stayed aligned with the frontend contract. Tyler maintained test coverage and QA rigor through each phase. Dias owned the data foundation and documentation quality that made the milestone documents coherent and complete. The team worked collaboratively, communicated consistently, and delivered a product that is both functional and portfolio-ready.

4. Post-Analysis: Lessons Learned

Written by the Team Lead. This section reflects on the major challenges encountered across the semester and proposes concrete improvements for future teams.

4.1 Major Technical Challenges

• Choosing SQLite for initial development was fast but became a concurrency bottleneck as the system grew. Future teams should evaluate PostgreSQL from day one — the migration cost later far exceeds the setup cost earlier.

• Integrating image analysis without a paid vision API required building a local heuristic pipeline (brightness, saturation, warmth bias, aspect ratio, tag inference). This taught us that deterministic signal extraction can be surprisingly effective when well-calibrated, but the calibration work is non-trivial and should be budgeted explicitly.

• Managing production deployment across EC2 with nginx, uvicorn, and a React SPA required careful path handling, proxy configuration, and HTTPS setup. Securing HTTPS with Let's Encrypt and DuckDNS added complexity but was essential for credibility in demos and evaluation.

4.2 Major Organizational Challenges

• Milestone deadlines compressed toward the end of the semester, making it harder to maintain documentation quality alongside active development. Future teams should write documentation continuously rather than in batches at milestone time.

• Git coordination across five developers required consistent branch and commit discipline. Early informal practices created merge debt that took real effort to resolve. Establish branching conventions and PR review requirements in week one.

4.3 Major Communication Challenges

• Keeping all members aligned on API contracts between frontend and backend required explicit coordination. A shared OpenAPI/Swagger spec document would have saved hours of debugging contract mismatches.

• Role boundaries were sometimes unclear early on. Defining ownership per feature area explicitly in the first week reduces duplication and missed coverage.

4.4 Concrete Improvements for Future Teams

1. Define your API contract before building either end — use OpenAPI/Swagger from day one and treat it as a living document.

2. Set up HTTPS and a real domain in Week 1, not Week 15. The 'Not Secure' warning erodes demo credibility and takes longer to fix under deadline pressure.

3. Write smoke tests as you ship each feature, not as a batch at milestone time. Test coverage is far easier to maintain incrementally.

4. Use PostgreSQL in production even in a class project — SQLite's concurrency limits are a real ceiling that will surface under any meaningful load.

5. Treat the README and documentation as living artifacts updated with every significant change. Documentation written after the fact is always incomplete.

CSC 648 / CSC 848 — Section 03

Milestone 1 — Version 2

Home4U

AI-Assisted, Budget-Aware Interior Style Matching Platform

Team Number: 4

Team Members & Roles:
Caleb Ponce — Team Lead / System Architecture
Tyler Morris — Backend & AI Integration
Christopher Quach — Frontend Development
Mason Lee — Data Modeling & Scoring Engine
Dias Almat — Technical Writer

Team Lead Email: cponce8@sfsu.edu
Date: February 19, 2026
Version: 2.0

REVISION HISTORY

Version	Date	Author(s)	Description
0.1	Feb 1, 2026	Team	Initial draft
1.0	Feb 8, 2026	Team	Version 1 submission
2.0	Feb 19, 2026	Team	Revised based on instructor feedback; completed use cases, diagrams, functional requirements, competitive analysis

---

1. Executive Summary

Home4U is a web-based interior style guidance platform designed to bridge the gap between visual inspiration and actionable transformation. Existing platforms such as Pinterest, Houzz, and Wayfair provide visual inspiration or shopping access, but they do not translate aesthetic styles into structured, budget-aware recommendations tailored to a user's actual space.

Home4U enables renters and first-time apartment dwellers to upload a room image, select desired interior styles (e.g., Modern, Japandi, Industrial), and receive AI-assisted tag suggestions describing current room characteristics. Using a weighted resemblance scoring algorithm, the system computes similarity between the user's room and the selected style(s). Based on this score, Home4U generates prioritized recommendations that maximize aesthetic impact within a defined budget.

Unlike competitors, Home4U integrates explainable scoring, human-validated AI tagging, multi-style comparison, and budget-aware prioritization within a single structured workflow.

The system architecture consists of a React frontend, FastAPI backend (Python 3.12), PostgreSQL relational database, and OpenAI Vision API for tag suggestion, deployed on AWS EC2 behind an Nginx reverse proxy secured with SSL/TLS.

---

2. Main Use Cases

2.1 Actors

- **User (Renter)**
- **Administrator**

2.2 Structured Use Cases

UC-01: Create Account

Primary Actor: User
Preconditions: User is not authenticated
Postconditions: User account is created and stored

Main Flow:

1. User selects “Sign Up”.
2. User enters email and password.
3. System validates input format.
4. System hashes password.
5. System stores user record.
6. System confirms account creation.

Alternative Flow:

- 3a: Email already exists → system displays error.

UC-02: Create Room Project

Primary Actor: User
Preconditions: User authenticated
Postconditions: RoomProject created

Main Flow:

1. User selects “Create Room”.
2. User enters room type.

3. System creates RoomProject record.
4. System confirms creation.

UC-03: Upload Room Image

Primary Actor: User
Preconditions: RoomProject exists
Postconditions: Image stored and linked

Main Flow:

1. User uploads image.
2. System validates file type.
3. System stores image reference.
4. System associates image with RoomProject.

UC-04: Select Target Style

Primary Actor: User
Preconditions: RoomProject exists
Postconditions: Style(s) selected

Main Flow:

1. User selects one or more styles.
2. System associates styles with RoomProject.

UC-05: Confirm Tags

Primary Actor: User
Preconditions: Image uploaded
Postconditions: Confirmed tags stored

Main Flow:

1. System generates suggested tags.
2. User reviews suggestions.
3. User edits or confirms tags.
4. System stores confirmed RoomTags.

UC-06: Compute Resemblance Score

Primary Actor: System
Preconditions: Confirmed tags exist
Postconditions: Score stored

Main Flow:

1. System retrieves StyleTag weights.
2. System compares RoomTags with StyleTags.
3. System calculates normalized score.
4. System stores ResemblanceScore.

UC-07: Compare Multiple Styles

Primary Actor: User
Preconditions: Multiple styles selected
Postconditions: Comparison displayed

Main Flow:

1. System computes score for each style.
2. System displays comparison view.

UC-08: Input Budget Constraint

Primary Actor: User
Preconditions: Score calculated
Postconditions: Budget stored

Main Flow:

1. User inputs budget.
2. System validates amount.
3. System stores budget constraint.

UC-09: Generate Recommendations

Primary Actor: System
Preconditions: Budget and score available
Postconditions: Recommendations displayed

Main Flow:

1. System identifies missing style tags.
2. System ranks improvements by impact-to-cost ratio.
3. System displays prioritized recommendations.

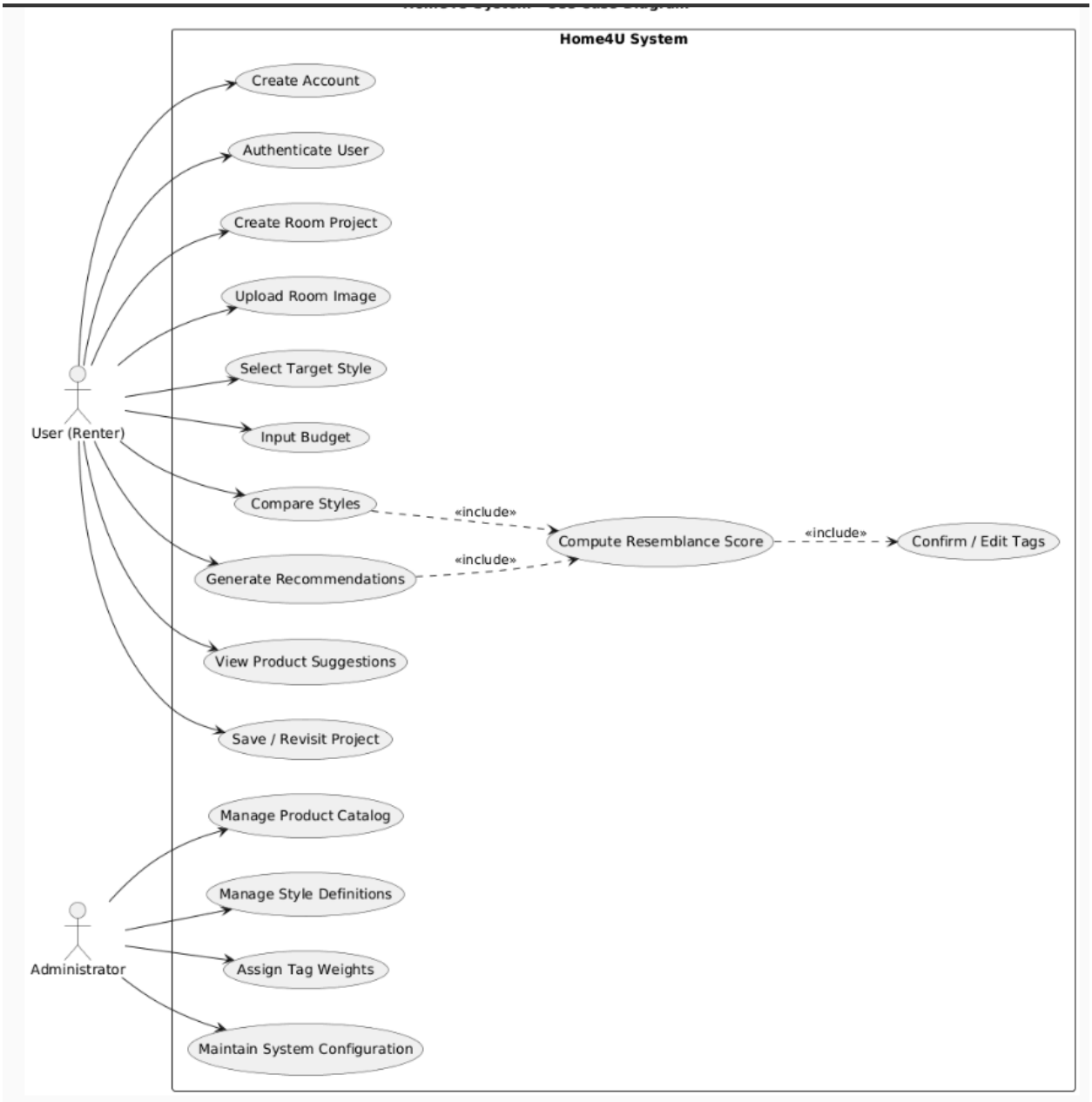
UC-10: Manage Style Definitions (Admin)

Primary Actor: Administrator
Preconditions: Admin authenticated
Postconditions: Style data updated

Main Flow:

1. Admin edits style definition.
2. Admin modifies tag weights.
3. System updates database.

2.3 Use Case Diagram



Actors:

- User
- Administrator

System boundary:

- Home4U System

Use cases inside system:

- Create Account
- Upload Image
- Select Style
- Confirm Tags
- Compute Score
- Generate Recommendations
- Manage Styles

---

3. Main Data Items and Entities

Each entity includes attributes and relationships.

3.1 User

- user_id (PK)
- email
- password_hash
- created_at

Relationship:

- One-to-many with RoomProject

3.2 RoomProject

- room_id (PK)
- user_id (FK)
- room_type
- budget

Relationship:

- One-to-many with RoomTag

3.3 Style

- style_id (PK)
- name
- description

3.4 Tag

- tag_id (PK)
- name

3.5 StyleTag

- style_id (FK)
- tag_id (FK)

- weight

3.6 RoomTag

- room_id (FK)
- tag_id (FK)

3.7 ResemblanceScore

- score_id (PK)
- room_id (FK)
- style_id (FK)
- score_value

3.8 Recommendation

- recommendation_id (PK)
- room_id (FK)
- description
- priority_score

3.9 ProductItem

- product_id (PK)
- style_id (FK)
- name
- estimated_cost
- url

---

4. Functional Requirements

4.1 User Requirements

USR-01: System shall allow user registration.
USR-02: System shall authenticate users securely.
USR-03: System shall allow creation of RoomProject.
USR-04: System shall allow image upload.
USR-05: System shall allow style selection.
USR-06: System shall allow tag confirmation.
USR-07: A user shall create many room projects.
USR-08: A user shall submit many feedback records.

4.2 RoomProject Requirements

RP-01: System shall create RoomProject records.
RP-02: System shall associate RoomProject with User.
RP-03: System shall store budget constraint.
RP-04: A room project shall belong to at most one user.
RP-05: A room project shall have at least one room image.
RP-06: A room project shall have at least one room dimension.
RP-07: A room project shall contain at least one room furniture record.
RP-08: A room project shall contain at least one room object record.
RP-09: A room project shall be associated with at least one room tag.
RP-10: A room project shall have at least one resemblance score.
RP-11: A room project shall have at most one budget plan.

4.3 Style Requirements

ST-01: System shall store predefined styles.
ST-02: Admin shall modify style definitions.
ST-03: A style shall be associated with many tags.
ST-04: A style shall have many resemblance scores.
ST-05: A style shall be associated with many product items.

4.4 Scoring Requirements

SC-01: System shall compute weighted resemblance score.
SC-02: System shall normalize scores to 0–100.
SC-03: System shall support multi-style comparison.

4.5 Recommendation Requirements

REC-01: System shall generate prioritized recommendations.
REC-02: System shall rank recommendations by impact-to-cost ratio.

4.6 Room Image Requirements

IMG-01: A room image shall belong to one room project.

4.7 Room Dimension Requirements

DIM-01: A room dimension record shall belong to exactly one room project.

4.8 Room Furniture Requirements

FUR-01: A room furniture record shall belong to exactly one room project.

4.9 Room Object Requirements

OBJ-01: A room object record shall belong to exactly one room project.
OBJ-02: A room object record shall be associated with at least one tag.

4.10 Tag Requirements

TAG-01: A tag shall be associated with many styles.
TAG-02: A tag shall be associated with many room projects.
TAG-03: A tag shall be associated with many room objects.

4.8 Product Item Requirements

ITM-01: A product item shall belong to exactly one vendor.
ITM-02: A vendor shall supply many product items.
ITM-03: A product item shall belong to exactly one product category.
ITM-04: A product category shall classify many product items.
ITM-05: A product item shall be associated with at most one style.
ITM-06: A product item shall appear in many recommendation items.
ITM-07: A product item shall be allocated in many budget allocations.

4.8 Vendor Requirements

VEN-01: A vendor shall supply many product items.
VEN-02: A product item shall belong to exactly one vendor.

4.8 Budget Plan Requirements

BUD-01: A budget plan shall belong to exactly one room project.
BUD-02: A budget plan shall be associated with at least one product item.

4.8 Feedback Requirements

FED-01: A feedback record shall belong to exactly one user.

---

5. Non-Functional Requirements

5.1 Performance

NFR-P-01: Scoring shall complete within 5 seconds.

5.2 Reliability

NFR-R-01: System shall maintain 99% uptime.

5.3 Security

NFR-S-01: Passwords shall be hashed and salted.
NFR-S-02: JWT authentication shall be implemented.
NFR-S-03: All communication shall occur over HTTPS using SSL/TLS.

5.4 Usability

NFR-U-01: Interface shall be responsive.

5.5 Scalability

NFR-SC-01: System shall support concurrent users.

5.6 Maintainability

NFR-M-01: Code shall follow PEP8 guidelines.
NFR-M-02: Functions shall include docstrings.

5.7 Coding Standards

CS-01: Backend shall use Python 3.12.
CS-02: REST endpoints shall follow consistent naming conventions.
CS-03: React 18 functional components required.
CS-04: Git branching strategy must be followed.
CS-05: No hardcoded secrets permitted.

---

6. Competitive Analysis

6.1 Table 1 – Feature Comparison

Product	Personalization	Style Scoring	Budget Filter	Shopping Integration
Pinterest	No	No	No	Indirect
Houzz	Limited	No	No	Yes
IKEA Planner	Limited	No	Limited	Yes
Havenly	Yes	No	No	Yes
Wayfair Ideas	Limited	No	No	Yes

6.2 Table 2 – Weakness & Opportunity Matrix

Competitor	Weakness	Opportunity for Home4U
Pinterest	No structured recommendations	Provide explainable scoring
Houzz	No scoring system	Add measurable similarity index
IKEA Planner	Store-specific	Multi-brand neutrality
Havenly	Paid service	Self-guided free alternative
Wayfair	No personalization engine	AI-assisted personalization

---

6.3 Analytical Summary

Existing platforms focus either on inspiration or commerce but lack explainable personalization. None provide a quantitative resemblance scoring system combined with budget-aware prioritization. Home4U differentiates itself by integrating structured scoring, AI-assisted tagging, and cost-impact ranking into a single user workflow.

6.4 Target Audience Validation

A preliminary validation exercise was conducted with six renters aged 21–30. Five reported difficulty translating visual inspiration into actionable steps within a fixed budget. Four expressed interest in a scoring system that quantifies style similarity. These findings support the need for structured, budget-aware personalization.

---

7. Project Readiness Checklist

☑ Meeting schedule established
☑ Roles assigned
☑ Tools selected
☑ Repository organized
☑ Python version standardized
☑ SSL planned

---

8. High-Level Architecture and Technologies

Frontend (React 18)
→ FastAPI (Python 3.12)
→ PostgreSQL 16
→ OpenAI Vision API
→ Nginx Reverse Proxy
→ AWS EC2 (Ubuntu 22.04 LTS)
→ SSL via Let’s Encrypt

---

9. Team Contributions

The team divided responsibilities based on technical strengths while ensuring collaborative review of all major deliverables. Each member was responsible for primary ownership of specific sections and technical components.

Caleb Ponce — Team Lead / System Architecture

Authored and finalized:

- Section 1: Executive Summary
- Section 8: High-Level Architecture and Technologies
- Architecture stack definition (React 18 → FastAPI → PostgreSQL → OpenAI Vision API → Nginx → AWS EC2 → SSL)

Defined and structured:

- All 10 Structured Use Cases (UC-01 through UC-10)
- System boundary and actor definitions in Section 2.1
- Section 2.3 Use Case Diagram (page 6)

Created Functional Requirements:

- SC-01, SC-02, SC-03 (Scoring Requirements)
- REC-01, REC-02 (Recommendation Requirements)
- NFR-S-01, NFR-S-02, NFR-S-03 (Security Requirements)
- CS-01 through CS-05 (Coding Standards)

Reviewed and approved:

- Entity structure consistency across Section 3

- Competitive differentiation summary (Section 6.3)

Tyler Morris — Backend & AI Integration

Defined backend-related use cases:

- UC-05 (Confirm Tags)
- UC-06 (Compute Resemblance Score)
- UC-09 (Generate Recommendations)
- UC-10 (Manage Style Definitions)

Authored Functional Requirements:

- USR-02 (User Authentication)
- SC-01 (Weighted resemblance scoring logic)
- SC-02 (Score normalization to 0–100)
- REC-01 (Recommendation generation logic)
- REC-02 (Impact-to-cost ranking algorithm)

Specified:

- OpenAI Vision API tag generation workflow
- FastAPI endpoint behavior for:
    - Image upload
    - Score computation
    - Recommendation generation
- JWT authentication requirement (NFR-S-02)

Christopher Quach — Frontend Development

Defined user-facing use cases:

- UC-01 (Create Account)
- UC-02 (Create Room Project)
- UC-03 (Upload Room Image)
- UC-04 (Select Target Style)
- UC-07 (Compare Multiple Styles)
- UC-08 (Input Budget Constraint)

Authored Functional Requirements:

- USR-01 (User registration)
- USR-03 (RoomProject creation)
- USR-04 (Image upload)
- USR-05 (Style selection)
- USR-06 (Tag confirmation interface)

Defined Non-Functional Requirements:

- NFR-U-01 (Responsive interface)
- CS-03 (React 18 functional component requirement)

Created:

- Frontend workflow alignment with use case diagram (page 6)
- Multi-style comparison UI specification

Mason Lee — Data Modeling & Scoring Engine

Authored Section 3 (Data Entities):

- 3.1 User
- 3.2 RoomProject
- 3.3 Style
- 3.4 Tag
- 3.5 StyleTag
- 3.6 RoomTag
- 3.7 ResemblanceScore
- 3.8 Recommendation
- 3.9 ProductItem

Defined relationships and key structures:

- PK/FK constraints
- Many-to-one and one-to-many mappings
- StyleTag weight attribute design
- ResemblanceScore entity structure

Created Functional Requirements:

- RP-01, RP-02, RP-03 (RoomProject Requirements)
- ST-01, ST-02 (Style Requirements)

Specified:

- Weighted scoring data schema
- Normalization storage model

Dias Almat — Technical Writer

Authored and formatted:

- Section 6: Competitive Analysis
    - Table 1 – Feature Comparison
    - Table 2 – Weakness & Opportunity Matrix
    - Section 6.3 Analytical Summary
    - Section 6.4 Target Audience Validation

Authored Non-Functional Requirements:

- NFR-P-01 (Performance – 5 second scoring requirement)
- NFR-R-01 (99% uptime requirement)
- NFR-SC-01 (Scalability requirement)
- NFR-M-01, NFR-M-02 (Maintainability requirements)

Prepared:

- Revision History table
- Project Readiness Checklist (Section 7)
- Document formatting, consistency review, and final submission compilation

CSC 648 / CSC 848

Section 03

Milestone 2 — Version 2 (M2v2)

Project: **Home4U**
AI-Assisted Interior Design Recommendation Platform

Team Number: **4**
Team Alias: **Vibecoding for Internship**

Team Members and Roles

Caleb Ponce — Team Lead, Backend/Auth, Deployment
Mason Lee — Frontend/UI
Christopher Quach — Backend/API & Database Alignment
Tyler Morris — QA / Smoke Tests / Deployment Verification
Dias Almat — Data (Styles/Tags/Search) & Recommendations

Team Lead Email

cponce8@sfsu.edu

Date

March 4, 2026

Revision History

Version	Date	Author(s)	Description
M2v2	Mar 5th, 2026	Caleb Ponce	Revised Milestone 2 submission, edited the team contributions/scores, made sure it met demands, and submitted a new one.
M2v1	Mar 4, 2026	Team	Final Milestone 2 submission
M2v0.9	Mar 3, 2026	Team	Completed architecture diagrams, APIs, and deployment documentation
M2v0.7	Mar 2, 2026	Team	Drafted data definitions and functional requirements
M2v0.5	Mar 1, 2026	Team	Created Milestone 2 document structure
M1v2	Feb 19, 2026	Team	Revised Milestone 1 based on instructor feedback
M1v1	Feb 8, 2026	Team	Initial Milestone 1 submission
M1v0.1	Feb 1, 2026	Team	Initial project draft

Table of Contents

‘

1. Data Definitions

The following entities represent the core database schema used by the Home4U backend.

User

Represents a registered user of the platform.

Attributes

user_id
email
password_hash
created_at

Relationship:

User → RoomProject

RoomProject

Represents a user-created room design project.

Attributes

project_id
user_id (FK → User)
room_type
budget
photo_url
created_at

Relationship:

RoomProject → RoomTag
RoomProject → Room Dimension
RoomProject → ResemblanceScore
RoomProject → Recommendation
RoomProject → RoomFurniture
RoomProject → RoomObject

Style

Represents an interior design style available for recommendation.

Attributes

style_id
name
description

Relationship:
Style → StyleTag
Style → ProductItem
Style → ResemblanceScore

Tag

Represents descriptive keywords used to categorize styles.

Attributes

tag_id
name

Relationship:
Tag → RoomTag
Tag → StyleTag

StyleTag

Associates tags with styles.

Attributes

styletag_id
style_id
tag_id
weight

RoomTag

Associates tags with a room project.

Attributes

roomtag_id
room_project_id
tag_id
is_confirmed

ResemblanceScore

Stores similarity scores between room projects and styles.

Attributes

resemblancescore_id
room_project_id
style_id
score_value
created_at

Recommendation

Stores design recommendations generated for a project.

Attributes

recommendation_id
room_project_id
description
priority_score
estimated_cost
is_completed
created_at

Relationship:
Recommendation → RoomProject

ProductItem

Represents product suggestions associated with a style.

Attributes

product_id
style_id
vendor_id
name
category
estimated_cost
url
Image_url

Relationship:
ProductItem → Style
ProductItem → Vendor
ProductItem → BudgetAllocation

BudgetPlan

Represents the budget associated with the project

Attributes

budget_id
project_id
total_budget
currency
plan_name
created_at

Relationship:
BudgetPlan → RoomProject
BudgetPlan → BudgetAllocation

BudgetAllocation

Represents how the budget is being spent

Attributes

allocation_id
budget_id
product_id
category
allocated_amount
quantity
priority_rank
created_at

Relationship:
BudgetAllocation → ProductItem
BudgetAllocation → BudgetPlan

---

2. Prioritized Functional Requirements

Priority 1 — Critical

FR1 — User Signup
Users shall be able to register using an email and password.

FR2 — User Login
Users shall authenticate and receive a JWT token.

FR3 — Create Room Project
Authenticated users shall create room design projects.

FR4 — List Room Projects
Users shall retrieve projects associated with their account.

FR5 — Retrieve Styles
Users shall retrieve available interior design styles.

FR6 — Retrieve Style Tags
Users shall retrieve tags associated with styles.

FR7 — Search Styles
Users shall search styles using text queries.

FR8 — Generate Recommendations
Users shall receive recommendations associated with a project.

FR9 — Enforce Project Ownership
The system shall ensure that users can access only the room projects associated with their own account.

FR10 — Validate User Credentials
The system shall validate user credentials before granting access to protected resources.

FR11 — Persist Room Project Data
The system shall store room project information in the database for later retrieval.

FR12 — Associate Recommendations with Styles
The system shall associate each generated recommendation with a style.

Priority 2 — Important

FR13 — Update Room Project
Users shall modify project information.

FR14 — Delete Room Project
Users shall delete projects.

FR15 — Upload Room Photo
Users shall upload a room image which is stored locally.

FR16 — Assign Tags to Room Project
Users shall associate tags with projects.

FR17 — Compute Resemblance Scores
The system shall calculate similarity scores between room tags and style tags.

FR18 — Mark Recommendation Complete
Users shall mark recommendations as completed.

FR19 — Update Recommendation Status
Users shall update the status of a recommendation.

FR20 — Retrieve Tags
Users shall retrieve the set of available tags.

FR21 — Search Tags
Users shall search tags using text queries.

FR22 — Associate Style Tags with Styles
The system shall associate tags with styles.

FR23 — Support Multiple Room Photos per Project
The system shall allow a room project to have multiple uploaded room photos.

Priority 3 — Nice to Have

FR24 — Product Suggestions
The system may recommend purchasable furniture products.

FR25 — AI Tag Suggestions
Future versions may include automated tag generation using computer vision.

---

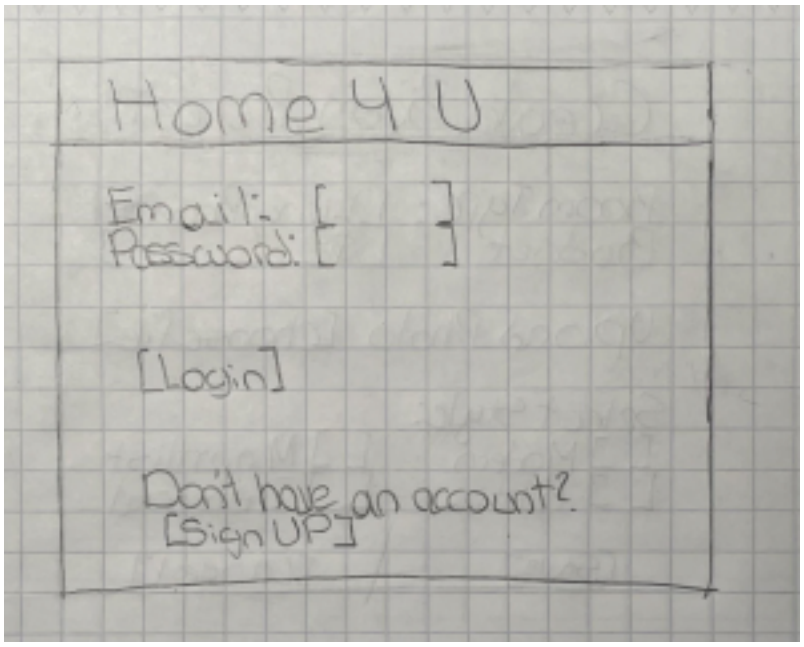
3. Mockups and Storyboards

This section presents simplified wireframe layouts representing the major pages of the Home4U platform.

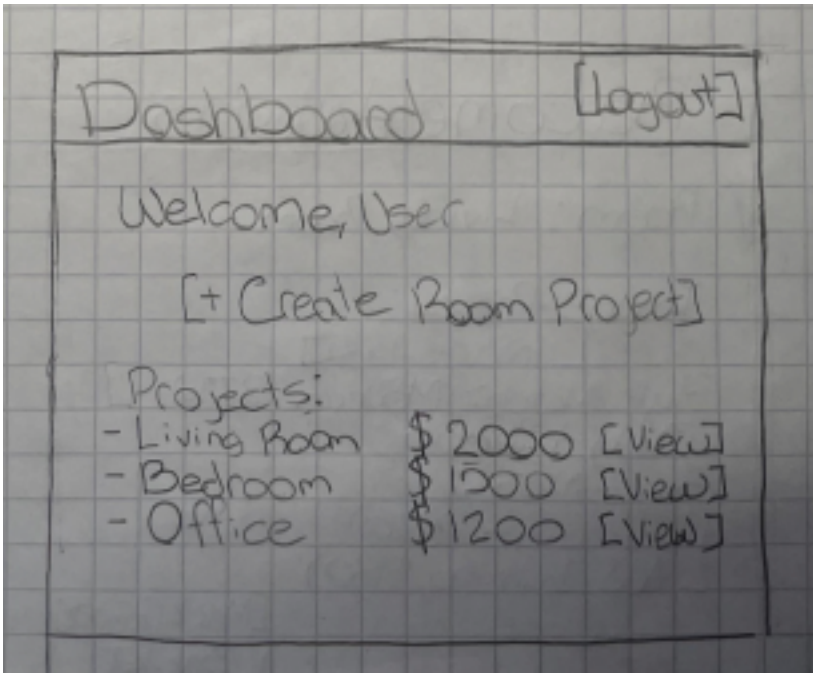
Primary User Workflow

Login
↓
Dashboard
↓
Create Room Project
↓
Upload Room Photo
↓
Search/Browse Styles
↓
View Recommendations

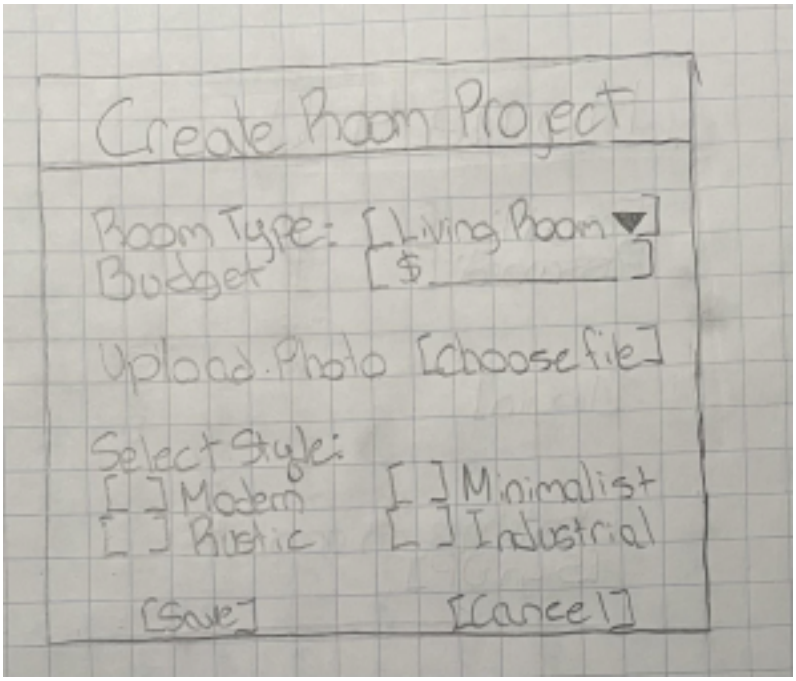
Login Page



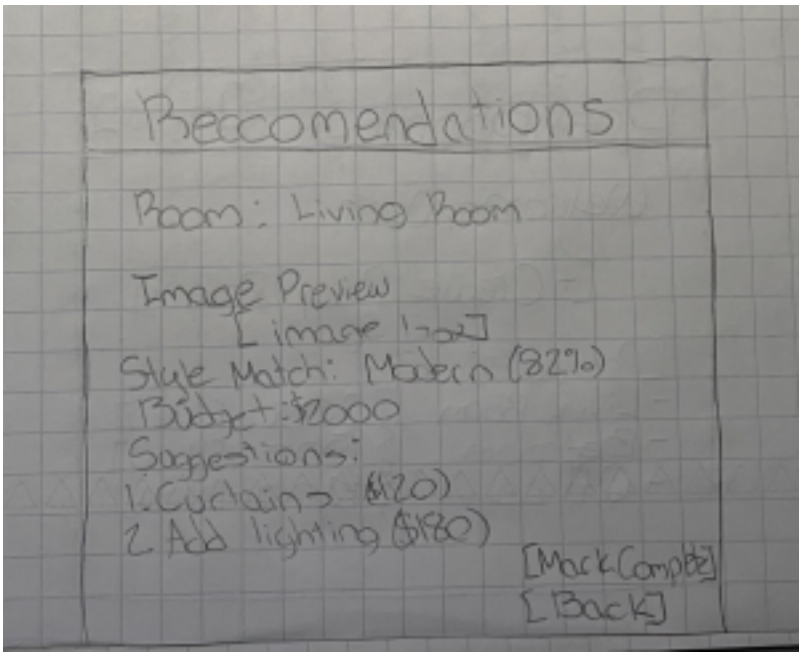
Dashboard



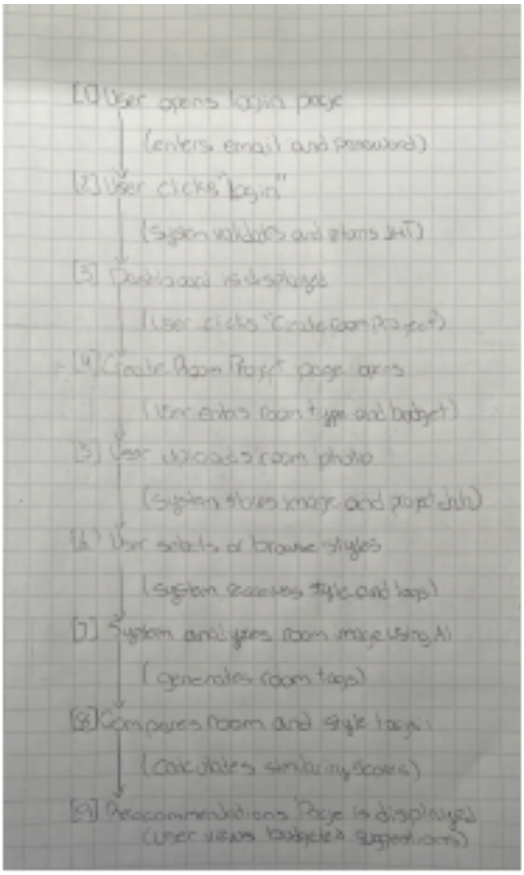
Create Room Project



Recommendations Page



Storyboard


[1] User opens login page
(enters email and password)
[2] User clicks "Login"
(System validates and starts JWT)
[3] Dashboard is displayed
(User clicks "Create Room Project")
[4] "Create Room Project" page opens
(User enters room type and budget)
[5] User uploads room photo
(System stores image and page data)
[6] User selects or browse styles
(System receives style and tags)
[7] System analyzes room image using AI
(generates room tags)
[8] Compares room and style tags
(calculates similarity scores)
[9] Recommendations Page is displayed
(User views ranked suggestions)

4. High-Level System Design

4.1 System Architecture Overview

Home4U follows a **client–server architecture** in which the frontend application communicates with the backend through REST APIs. The architecture separates the user interface, application logic, and data storage layers to improve scalability and maintainability.

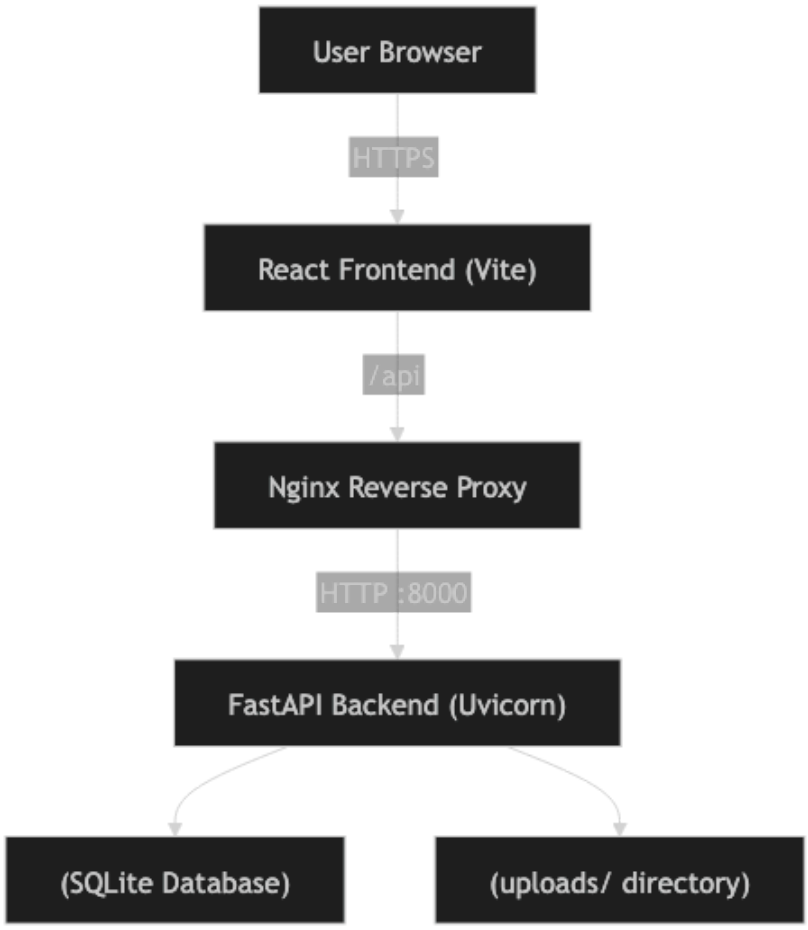
The frontend application is implemented using **React with the Vite build system**, which communicates with the backend through HTTP requests. These requests are routed through **Nginx**, which acts as a reverse proxy and handles routing and request forwarding to the FastAPI backend.

The backend service is implemented using **FastAPI**, a high-performance Python web framework. The backend processes requests, performs authentication using JWT tokens, executes search and recommendation algorithms, and communicates with the database.

Data persistence for the vertical prototype is handled using **SQLite**, which stores user data, projects, styles, tags, and recommendations. Uploaded room images are stored in a local **uploads directory** referenced by the database.

Future versions of the system will migrate the database to **PostgreSQL** and store media files in **AWS S3** to improve scalability and reliability.

Figure 1 — Home4U System Architecture



4.2 Backend Technologies

The backend of the Home4U system is implemented using modern web technologies designed for scalability and maintainability.

Backend Framework

FastAPI (Python) is used to implement the REST API services. FastAPI provides automatic request validation, asynchronous support, and high performance.

Authentication

User authentication is implemented using:

- bcrypt password hashing
- JWT authentication tokens (HS256)
- case-insensitive email normalization

These mechanisms ensure that user credentials are securely stored and verified.

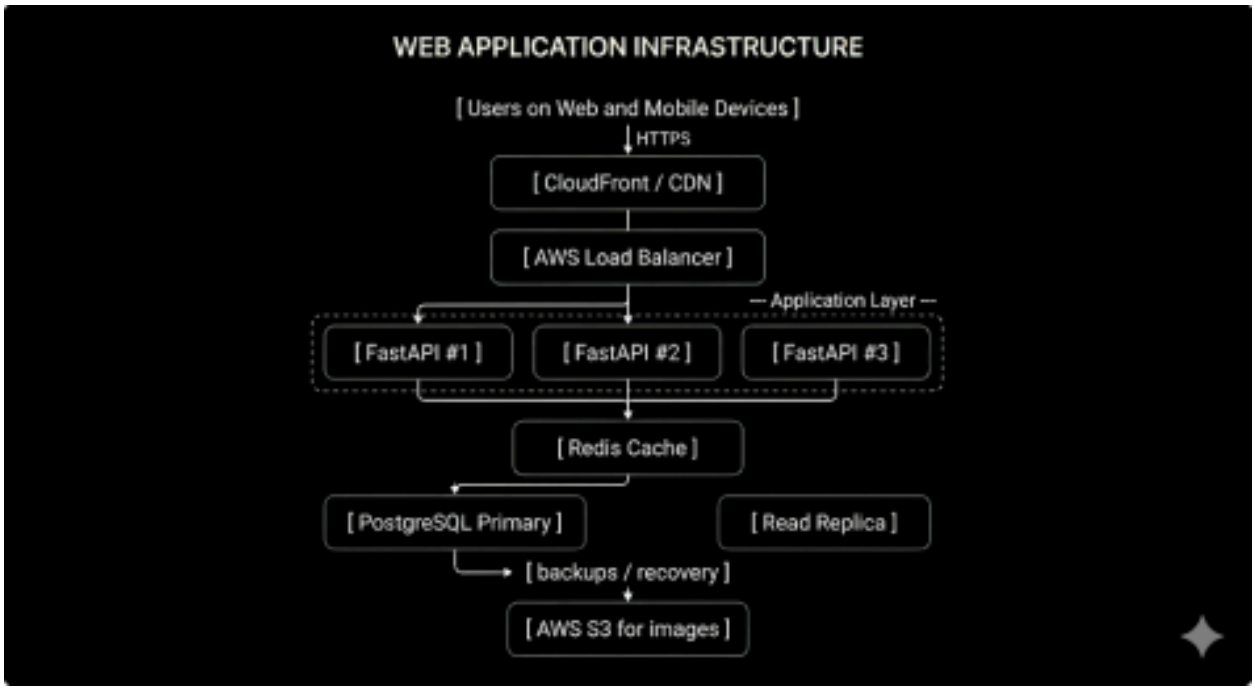
Database

For the vertical prototype, the system uses **SQLite**, which allows rapid development and simplified deployment. SQLite stores all core entities including users, projects, styles, tags, and recommendations.

Future production versions will migrate to **PostgreSQL**, which provides stronger concurrency support and improved scalability.

Media Storage

Room images uploaded by users are stored locally in an **uploads directory** on the server. The database stores a reference to the image location using the `photo_url` field.



Scalability Architecture Plan.

For the current vertical prototype, Home4U can run on a single backend instance. As traffic increases, the system can scale horizontally by placing a load balancer in front of multiple FastAPI application servers. User-uploaded room images should be moved from local server storage to AWS S3 so that media remains available across all backend instances. The database should migrate from SQLite to PostgreSQL because PostgreSQL provides stronger concurrency control, better support for multiple simultaneous users, and easier replication for reliability. A Redis cache can be added to reduce repeated database reads for frequently accessed style, tag, and recommendation data. This design allows the system to grow from a single-instance prototype to a multi-instance production deployment while improving performance, reliability, and fault tolerance.

4.3 Network and Deployment Design

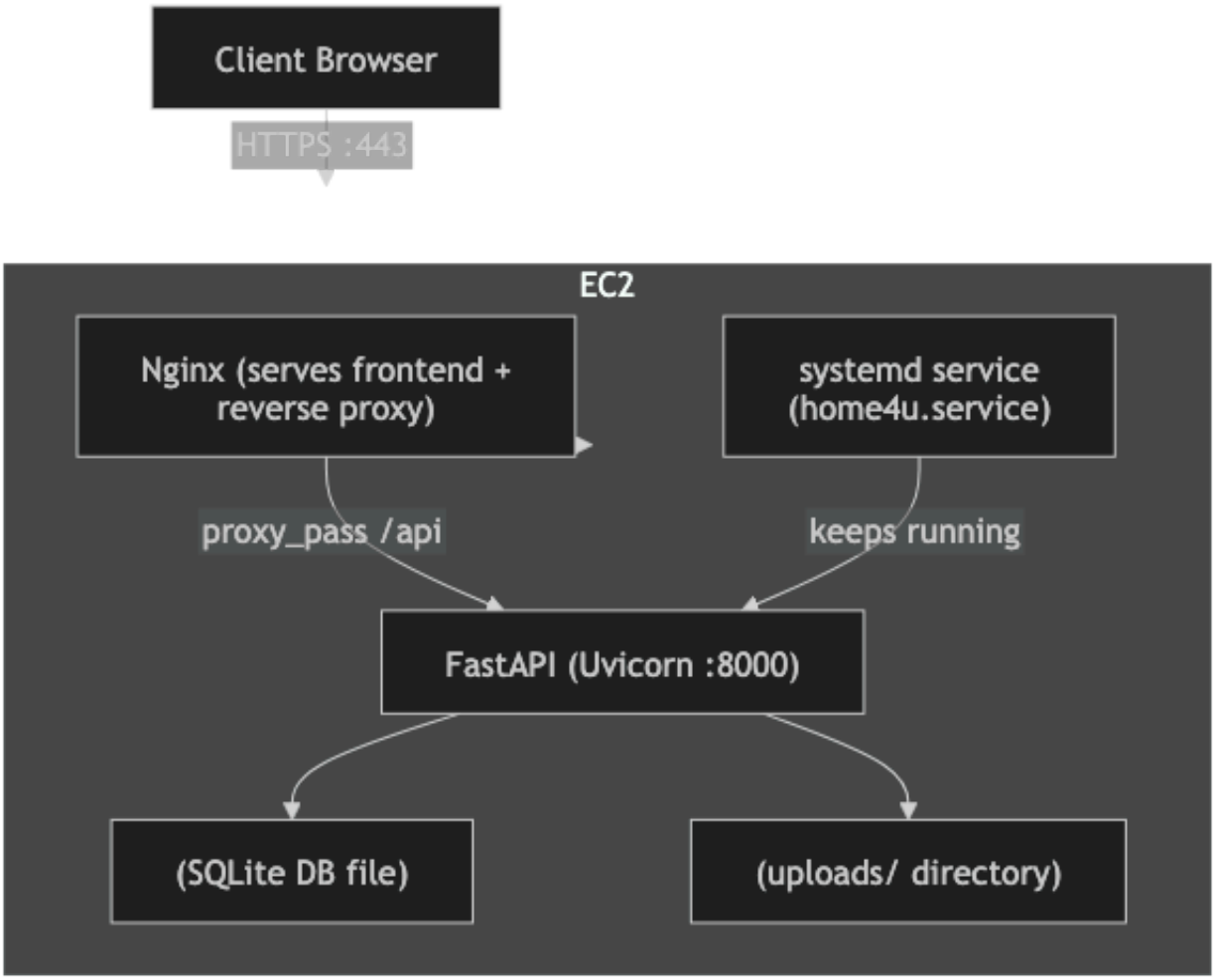
The Home4U platform is deployed on an **AWS EC2 instance running Ubuntu**. The deployment architecture uses a reverse proxy configuration to separate web traffic from application logic.

Incoming HTTP or HTTPS requests are received by **Nginx**, which serves the frontend application and forwards API requests to the FastAPI backend running on port 8000.

The backend service is managed by a **systemd service**, which ensures that the FastAPI server automatically restarts if the process fails or if the system reboots.

This architecture provides a simple but reliable infrastructure suitable for the vertical prototype while allowing future scaling.

Figure 2 — Home4U Deployment Architecture



4.4 High-Level APIs and Algorithms

The system uses the OpenAI Vision API to analyze a user's uploaded room images, using the api it then analyzes the user current room and generates descriptive tags representing the rooms visual features such as furniture types, colors, layout, and lighting. These tags are reviewed and validated before being used as structured inputs for style comparison and recommendation generation.

The weighted resemblance scoring algorithm is used to evaluate the similarity between a user's room and selected interior styles. The algorithm compares validated room tags against predefined style characteristics for each specific style such as modern,industrial, etc. Each matching feature contributes to a weighted score based on its visual importance, producing an overall similarity value that reflects how closely the room matches each selected style.

This similarity score is used to determine which styles most closely align with the current room configuration and to guide the generation of targeted improvement recommendations.

The system uses a budget-aware prioritization algorithm to generate recommendations that remain within a user-defined spending limit. Based on the calculated similarity scores, the system identifies potential room improvements that increase resemblance to the selected style. Each recommendation has an estimated cost to ensure that we do not go over the predefined budget if applicable.

4.5 Database Architecture

The Home4U database schema is designed using a normalized relational structure. The schema includes entities for users, room projects, styles, tags, recommendations, and product items.

Join tables such as **RoomTag** and **StyleTag** are used to represent many-to-many relationships between projects, styles, and tags.

Similarity between room projects and styles is represented using the **ResemblanceScore** entity, which stores computed similarity scores used by the recommendation system.

Initial Database Requirements

The database design is driven by the Priority 1 functional requirements of the system, including user authentication, room project creation, project retrieval, style browsing, and recommendation generation. The database must support storing user accounts, associating users with their room projects, and maintaining relationships between room features and design styles through tags. It must also support retrieving recommendations based on computed similarity scores while ensuring that each user can only access their own data.

DBMS Selection and Justification

For the current vertical prototype, the system uses SQLite due to its simplicity, lightweight setup, and ease of integration during development. SQLite allows rapid iteration without requiring
complex configuration. However, for production deployment, the system will migrate to PostgreSQL. PostgreSQL is better suited for handling concurrent users, larger datasets, and more complex queries. It also provides stronger data integrity, indexing capabilities, and support for replication, making it more appropriate for a scalable production environment.

Media Storage Strategy

In the current implementation, uploaded room images are stored locally in an uploads directory on the server, and their file paths are stored in the database. While this approach is sufficient

for a single-instance prototype, it does not scale well in distributed environments. In future versions, media storage will be migrated to AWS S3, which provides scalable, durable, and centralized storage accessible by multiple backend instances. This change will improve system reliability and support horizontal scaling.

Data Integrity and Relationships

The database enforces relationships between entities using foreign keys to maintain data consistency. For example, each RoomProject is associated with a valid User, and recommendations are linked to specific projects. Join tables such as RoomTag and StyleTag ensure proper mapping between tags, styles, and room projects. These relationships support accurate similarity scoring and recommendation generation while maintaining referential integrity across the system.

Future Improvements

Future enhancements to the database architecture include adding indexing to frequently queried fields such as style names and tags, implementing caching mechanisms for faster retrieval of recommendation data, and enabling database replication for improved availability and fault tolerance. These improvements will allow the system to handle increased user traffic and data volume more efficiently.

Entity Relationship Diagram

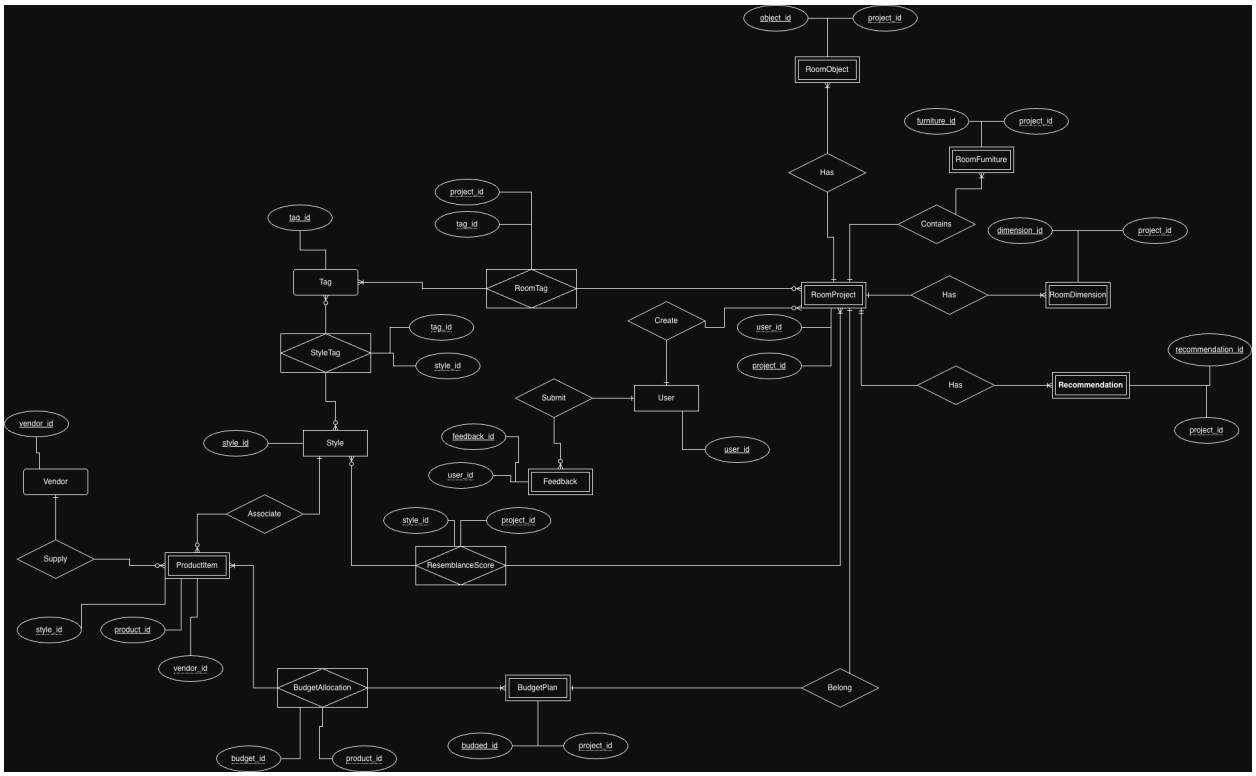


Figure 3 — Home4U Database Entity Relationship Diagram

5. Key Project Risks

Several potential risks could impact the development and deployment of the Home4U system.

Data Consistency Risks
Differences between frontend data handling and backend database schemas could result in inconsistent data states. This risk is mitigated by enforcing consistent API contracts and validating data using backend schemas.

Authentication Security Risks
Improper handling of authentication tokens could expose the system to security vulnerabilities. To mitigate this risk, the system uses bcrypt password hashing and JWT tokens with expiration times.

Single-Instance Infrastructure Risk
The vertical prototype currently runs on a single EC2 instance using SQLite. This creates a potential risk of downtime or data loss. Future versions will migrate to PostgreSQL and distributed cloud infrastructure.

Media Storage Growth
As more users upload room images, storage requirements will increase. Future versions of the system will migrate media storage to scalable cloud storage services such as AWS S3.

Team Coordination Risk
As multiple team members contribute to the same codebase, integration issues may arise. This risk is mitigated by using GitHub version control, pull requests, and regular team meetings.

---

6. Project Management

Development Tools

GitHub — version control
Notion — project tracking
Discord — team communication

Development Process

Trunk-based development with small pull requests and code reviews.

Definition of Done

Feature merged to repository
Deployment successful
Smoke tests pass
Core workflow operational

---

7. Team Contributions

Caleb Ponce — Team Lead, Backend/Auth, Deployment (Score: 10/10)

Caleb Ponce served as the Team Lead and ensured alignment across all milestone requirements, coordinating both technical execution and documentation quality.

On the technical side, Caleb implemented and maintained the connectivity between the frontend (React/Vite) and backend (FastAPI), ensuring reliable API communication across authentication and project workflows. He also refined the UI/UX of the platform to improve usability, navigation flow, and overall user experience, making the interface more intuitive and production-ready. Additionally, Caleb debugged and resolved frontend issues that impacted functionality and user interaction.

Beyond implementation, Caleb oversaw system integration, ensured consistency across diagrams and architecture, and supported team members to maintain a cohesive and polished final submission.

Mason Lee — Frontend/UI Development (Score: 10/10)

Mason Lee led the development of the frontend interface and collaborated closely on design architecture.

He worked with Christopher to design and refine frontend mockups and storyboards, ensuring consistency between UI components and system functionality. Mason contributed to structuring the layout of key pages such as authentication views, dashboard, and project interfaces, while aligning frontend behavior with backend expectations.

His work ensured that the frontend design was not only visually structured but also aligned with the overall system architecture and user flow.

Christopher Quach — Frontend Architecture & Backend Alignment (Score: 10/10)

Christopher Quach collaborated with Mason on frontend architecture, focusing on aligning system design with implementation.

He contributed to developing mockups and storyboards while ensuring that frontend structures were consistent with backend API design and database relationships. Christopher played a key

role in bridging frontend and backend logic, ensuring that components, data flow, and system interactions were coherent.

His contributions strengthened the architectural consistency between UI design and backend systems.

Tyler Morris — QA, Feedback Implementation, and Diagram Refinement (Score: 10/10)

Tyler Morris focused on quality assurance and directly implemented professor feedback to improve the project.

He revised and corrected system diagrams, specifically refining the Nginx reverse proxy and backend architecture diagrams to better reflect actual system behavior and deployment flow. Tyler ensured that diagrams were accurate, consistent, and aligned with the implemented architecture.

Additionally, he validated system functionality and ensured that revisions addressed prior feedback, improving the overall quality and correctness of the milestone.

Dias Almat — Data Design, ERD Refinement, and Diagram Quality Control (Score: 10/10)

Dias Almat played a critical role in refining the data architecture and improving documentation quality.

He revised and corrected the ERD diagram, ensuring accurate representation of entity relationships such as Style, Tag, RoomProject, and their associative tables. Dias also helped restructure architectural components where necessary to better reflect system design.

Beyond his individual contributions, he supported other team members by reviewing and improving diagrams across sections, ensuring visual consistency, clarity, and correctness throughout the documentation.

CSC 648 / CSC 848

Section 03

Milestone 3 — Version 2 (M3v2)

Project: **Home4U**
AI-Assisted Interior Design Recommendation Platform

Team Number: **4**
Team Alias: **Vibecoding for Internship**

Team Members and Roles

Caleb Ponce — Team Lead, Backend/Auth, Deployment
Mason Lee — Frontend/UI
Christopher Quach — Backend/API & Database Alignment
Tyler Morris — QA / Smoke Tests / Deployment Verification
Dias Almat — Data (Styles/Tags/Search) & Recommendations

Team Lead Email

cponce8@sfsu.edu

Date

April 14, 2026

Revision History

Version	Date	Author(s)	Description
M3v2	April 29, 2026	Team	Implemented figma designs to showcase our UI/UX correctly.
M3v2	April 21, 2026	Team	Redid diagrams to match frontend designs. Reworked wireframes.
M3v1	April 14th, 2026	Dias, Team	Added diagrams, design patterns
M2v2	Mar 5th, 2026	Caleb Ponce	Revised Milestone 2 submission, edited the team contributions/scores, made sure it met demands, and submitted a new one.
M2v1	Mar 4, 2026	Team	Final Milestone 2 submission
M2v0.9	Mar 3, 2026	Team	Completed architecture diagrams, APIs, and deployment documentation
M2v0.7	Mar 2, 2026	Team	Drafted data definitions and functional requirements
M2v0.5	Mar 1, 2026	Team	Created Milestone 2 document structure
M1v2	Feb 19, 2026	Team	Revised Milestone 1 based on instructor feedback
M1v1	Feb 8, 2026	Team	Initial Milestone 1 submission
M1v0.1	Feb 1, 2026	Team	Initial project draft

Table of Contents

1. Data Definitions

User

Represents a registered user of the platform.

Attributes

user_id
email
password_hash
created_at

Relationship:

User → RoomProject

RoomProject

Represents a user-created room design project.

Attributes

project_id
user_id (FK → User)
room_type
budget
photo_url
created_at

Relationship:

RoomProject → RoomTag
RoomProject → Room Dimension
RoomProject → ResemblanceScore
RoomProject → Recommendation
RoomProject → RoomFurniture
RoomProject → RoomObject

Style

Represents an interior design style available for recommendation.

Attributes

style_id
name
description

Relationship:
Style → StyleTag
Style → ProductItem
Style → ResemblanceScore

Tag

Represents descriptive keywords used to categorize styles.

Attributes

tag_id
name

Relationship:
Tag → RoomTag
Tag → StyleTag

StyleTag

Associates tags with styles.

Attributes

styletag_id
style_id
tag_id
weight

RoomTag

Associates tags with a room project.

Attributes

roomtag_id
room_project_id
tag_id
is_confirmed

ResemblanceScore

Stores similarity scores between room projects and styles.

Attributes

resemblancescore_id
room_project_id
style_id
score_value
created_at

Recommendation

Stores design recommendations generated for a project.

Attributes

recommendation_id
room_project_id
description
priority_score
estimated_cost
is_completed
created_at

Relationship:
Recommendation → RoomProject

ProductItem

Represents product suggestions associated with a style.

Attributes

product_id
style_id
vendor_id
name
category
estimated_cost
product_url
Image_url

Relationship:
ProductItem → Style
ProductItem → Vendor
ProductItem → BudgetAllocation

BudgetPlan

Represents the budget associated with the project

Attributes

budget_id
project_id
total_budget
currency
plan_name
created_at

Relationship:
BudgetPlan → RoomProject
BudgetPlan → BudgetAllocation

BudgetAllocation

Represents how the budget is being spent

Attributes

allocation_id

budget_id
product_id
category
allocated_amount
quantity
priority_rank
created_at

Relationship:
BudgetAllocation → ProductItem
BudgetAllocation → BudgetPlan

---

2. Prioritized Functional Requirements

Priority 1 — Critical

FR1 — User Signup
Users shall be able to register using an email and password.

FR2 — User Login
Users shall authenticate and receive a JWT token.

FR3 — Create Room Project
Authenticated users shall create room design projects.

FR4 — List Room Projects
Users shall retrieve projects associated with their account.

FR5 — Retrieve Styles
Users shall retrieve available interior design styles.

FR6 — Retrieve Style Tags
Users shall retrieve tags associated with styles.

FR7 — Search Styles
Users shall search styles using text queries.

FR8 — Generate Recommendations
Users shall receive recommendations associated with a project.

FR9 — Enforce Project Ownership
The system shall ensure that users can access only the room projects associated with their own account.

FR10 — Validate User Credentials
The system shall validate user credentials before granting access to protected resources.

FR11 — Persist Room Project Data
The system shall store room project information in the database for later retrieval.

FR12 — Associate Recommendations with Styles
The system shall associate each generated recommendation with a style.

Priority 2 — Important

FR13 — Update Room Project
Users shall modify project information.

FR14 — Delete Room Project
Users shall delete projects.

FR15 — Upload Room Photo
Users shall upload a room image which is stored locally.

FR16 — Assign Tags to Room Project
Users shall associate tags with projects.

FR17 — Compute Resemblance Scores
The system shall calculate similarity scores between room tags and style tags.

FR18 — Mark Recommendation Complete
Users shall mark recommendations as completed.

FR19 — Update Recommendation Status
Users shall update the status of a recommendation.

FR20 — Retrieve Tags
Users shall retrieve the set of available tags.

FR21 — Search Tags
Users shall search tags using text queries.

FR22 — Associate Style Tags with Styles
The system shall associate tags with styles.

FR23 — Support Multiple Room Photos per Project
The system shall allow a room project to have multiple uploaded room photos.

Priority 3 — Nice to Have

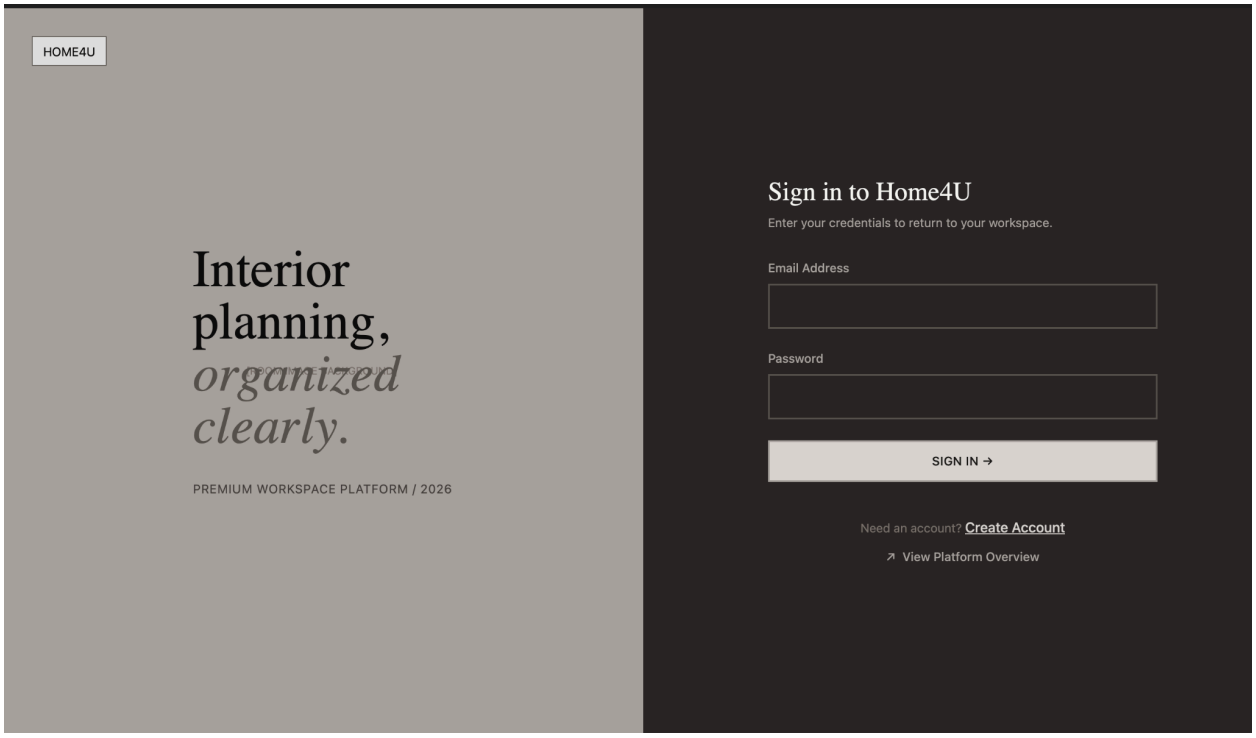
FR24 — Product Suggestions
The system may recommend purchasable furniture products.

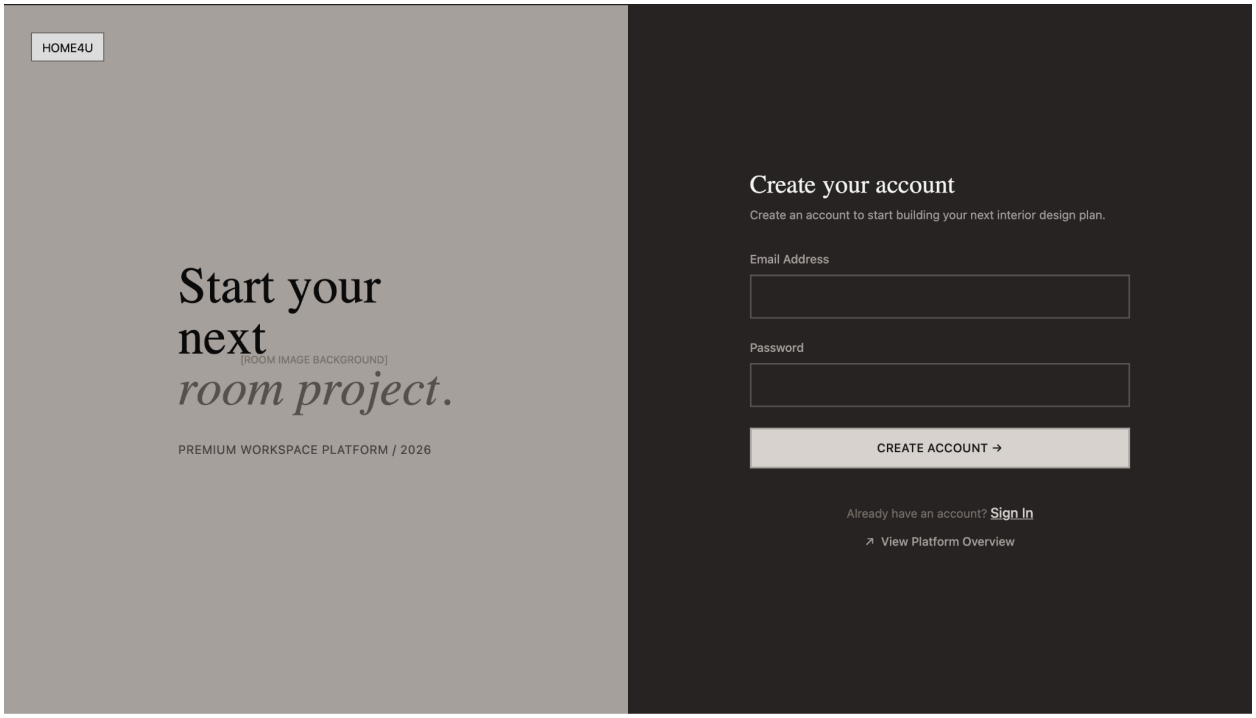
FR25 — AI Tag Suggestions
Future versions may include automated tag generation using computer vision.

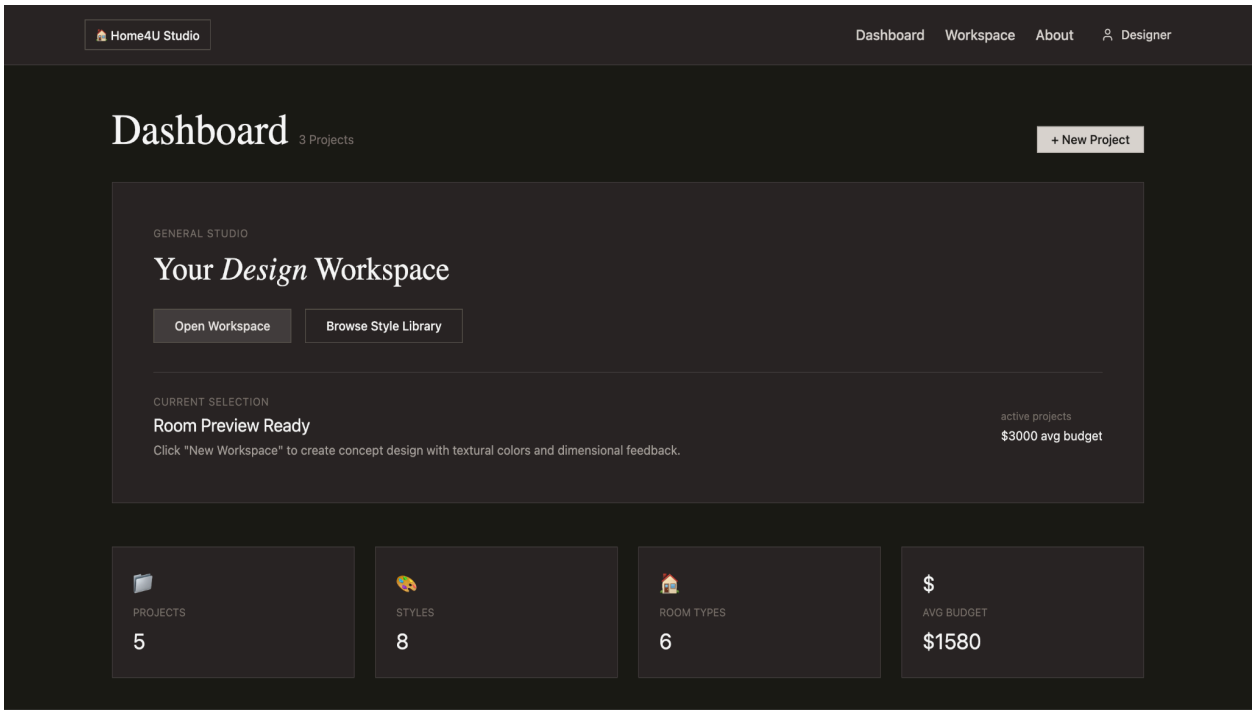
---

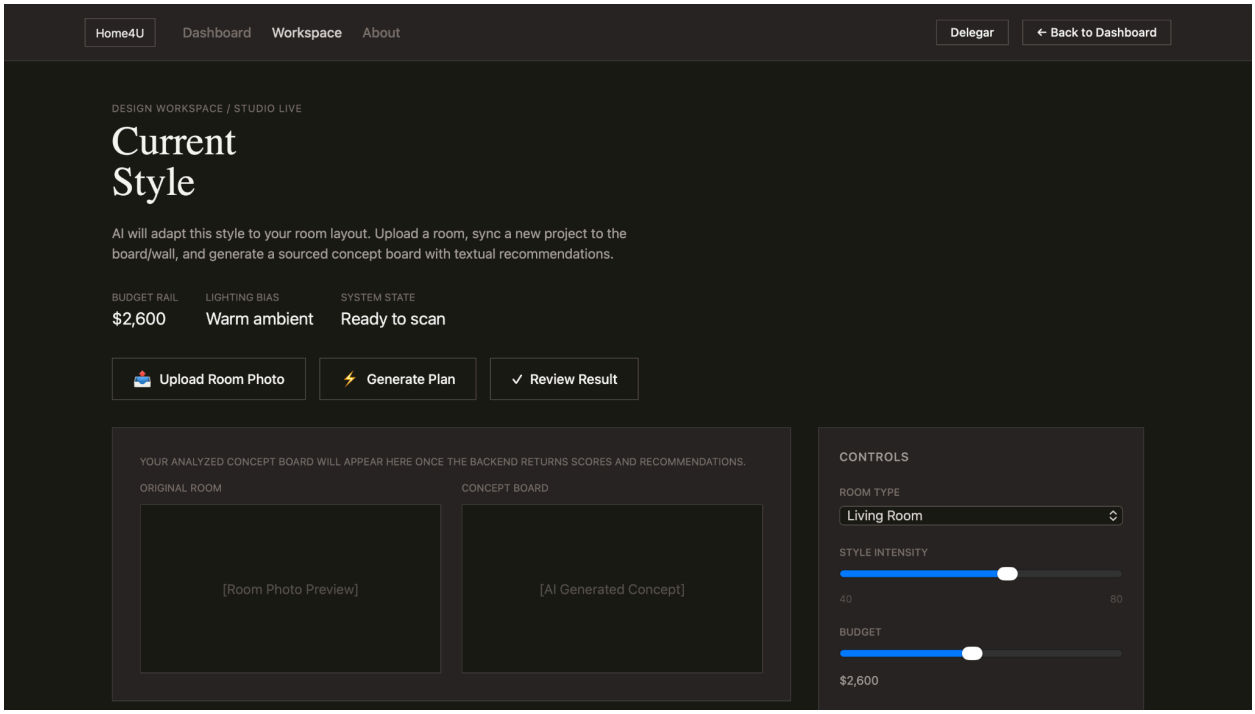
3. UI/UX Wireframes

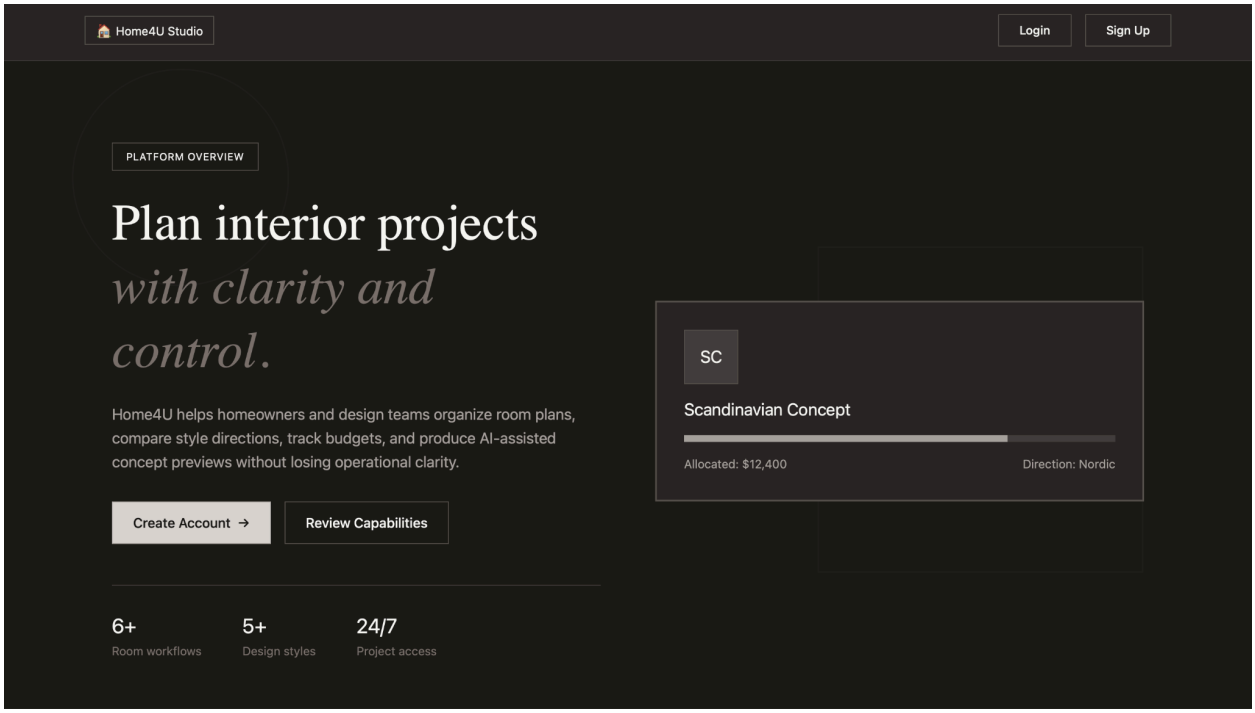
https://www.figma.com/make/InEql8vLWuBoS01xvxlEks/home4u--Community-?fullscreen=1&t=CY7lR23Epj7mnoFl-1




HOME4U
Start your next
[ROOM IMAGE BACKGROUND]
room project.
PREMIUM WORKSPACE PLATFORM / 2026
Create your account
Create an account to start building your next interior design plan.
Email Address
Password
CREATE ACCOUNT →
Already have an account? Sign In
↗ View Platform Overview


Home4U Studio
Dashboard
Workspace
About
Designer
Dashboard
3 Projects
+ New Project
GENERAL STUDIO
Your Design Workspace
Open Workspace
Browse Style Library
CURRENT SELECTION
Room Preview Ready
Click "New Workspace" to create concept design with textural colors and dimensional feedback.
active projects
$3000 avg budget
PROJECTS
5
STYLES
8
ROOM TYPES
6
$
AVG BUDGET
$1580


Home4U
Dashboard
Workspace
About
Delegar
← Back to Dashboard
DESIGN WORKSPACE / STUDIO LIVE
Current Style
AI will adapt this style to your room layout. Upload a room, sync a new project to the board/wall, and generate a sourced concept board with textual recommendations.
BUDGET RAIL
$2,600
LIGHTING BIAS
Warm ambient
SYSTEM STATE
Ready to scan
Upload Room Photo
Generate Plan
Review Result
YOUR ANALYZED CONCEPT BOARD WILL APPEAR HERE ONCE THE BACKEND RETURNS SCORES AND RECOMMENDATIONS.
ORIGINAL ROOM
[Room Photo Preview]
CONCEPT BOARD
[AI Generated Concept]
CONTROLS
ROOM TYPE
Living Room
STYLE INTENSITY
40
80
BUDGET
$2,600


Home4U Studio
Login
Sign Up
PLATFORM OVERVIEW
Plan interior projects with clarity and control.
Home4U helps homeowners and design teams organize room plans, compare style directions, track budgets, and produce AI-assisted concept previews without losing operational clarity.
Create Account →
Review Capabilities
6+
Room workflows
5+
Design styles
24/7
Project access
SC
Scandinavian Concept
Allocated: $12,400
Direction: Nordic

4. High-Level System Design

4.1 Database Architecture

The Home4U database schema is designed using a normalized relational structure. The schema includes entities for users, room projects, styles, tags, recommendations, and product items.

Initial Database Requirements

The database design is driven by the Priority 1 functional requirements of the system, including user authentication, room project creation, project retrieval, style browsing, and recommendation generation. The database must support storing user accounts, associating users with their room projects, and maintaining relationships between room features and design styles through tags. It must also support retrieving recommendations based on computed similarity scores while ensuring that each user can only access their own data.

DBMS Selection and Justification

For the current vertical prototype, the system uses SQLite due to its simplicity, lightweight setup, and ease of integration during development. SQLite allows rapid iteration without requiring
complex configuration. However, for production deployment, the system will migrate to PostgreSQL. PostgreSQL is better suited for handling concurrent users, larger datasets, and more complex queries. It also provides stronger data integrity, indexing capabilities, and support for replication, making it more appropriate for a scalable production environment.

Media Storage Strategy

In the current implementation, uploaded room images are stored locally in an uploads directory on the server, and their file paths are stored in the database. While this approach is sufficient for a single-instance prototype, it does not scale well in distributed environments. In future versions, media storage will be migrated to AWS S3, which provides scalable, durable, and centralized storage accessible by multiple backend instances. This change will improve system reliability and support horizontal scaling.

Data Integrity and Relationships

The database enforces relationships between entities using foreign keys to maintain data consistency. For example, each RoomProject is associated with a valid User, and recommendations are linked to specific projects. Join tables such as RoomTag and StyleTag ensure proper mapping between tags, styles, and room projects. These relationships support

accurate similarity scoring and recommendation generation while maintaining referential integrity across the system.

Future Improvements

Future enhancements to the database architecture include adding indexing to frequently queried fields such as style names and tags, implementing caching mechanisms for faster retrieval of recommendation data, and enabling database replication for improved availability and fault tolerance. These improvements will allow the system to handle increased user traffic and data volume more efficiently.

Entity Relationship Diagram

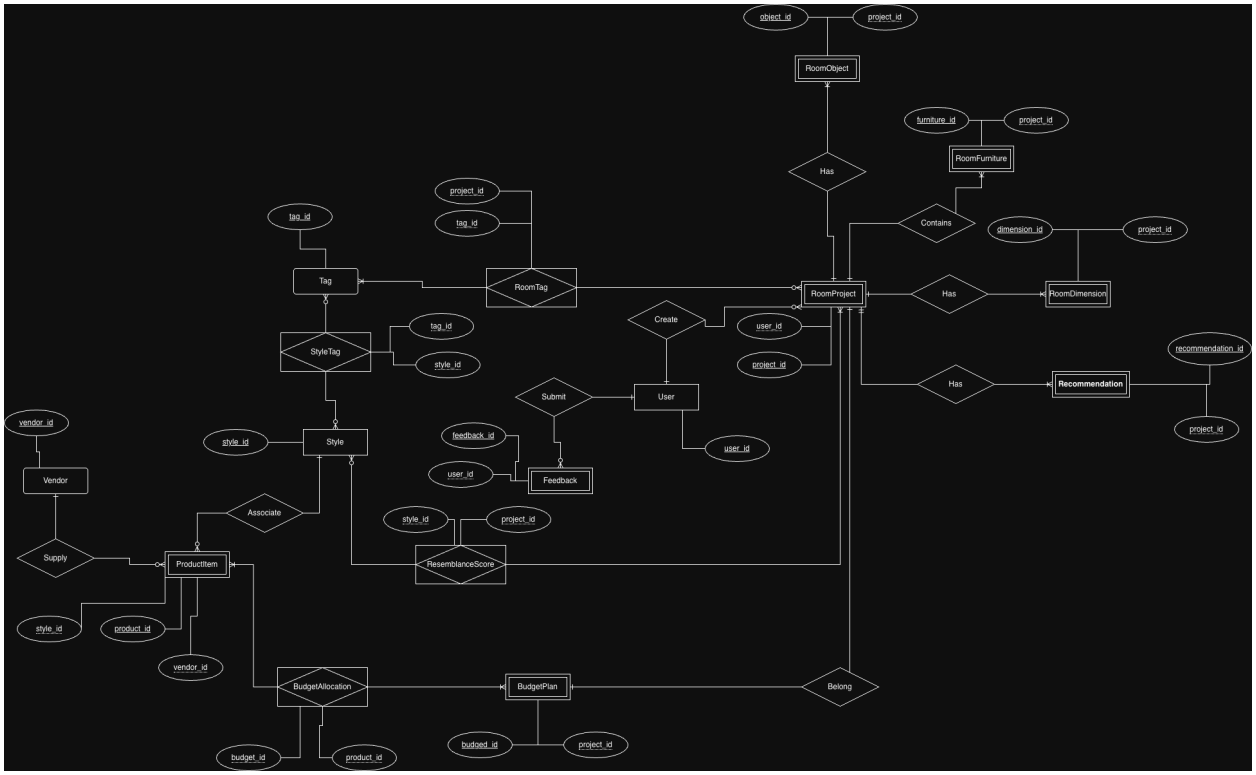
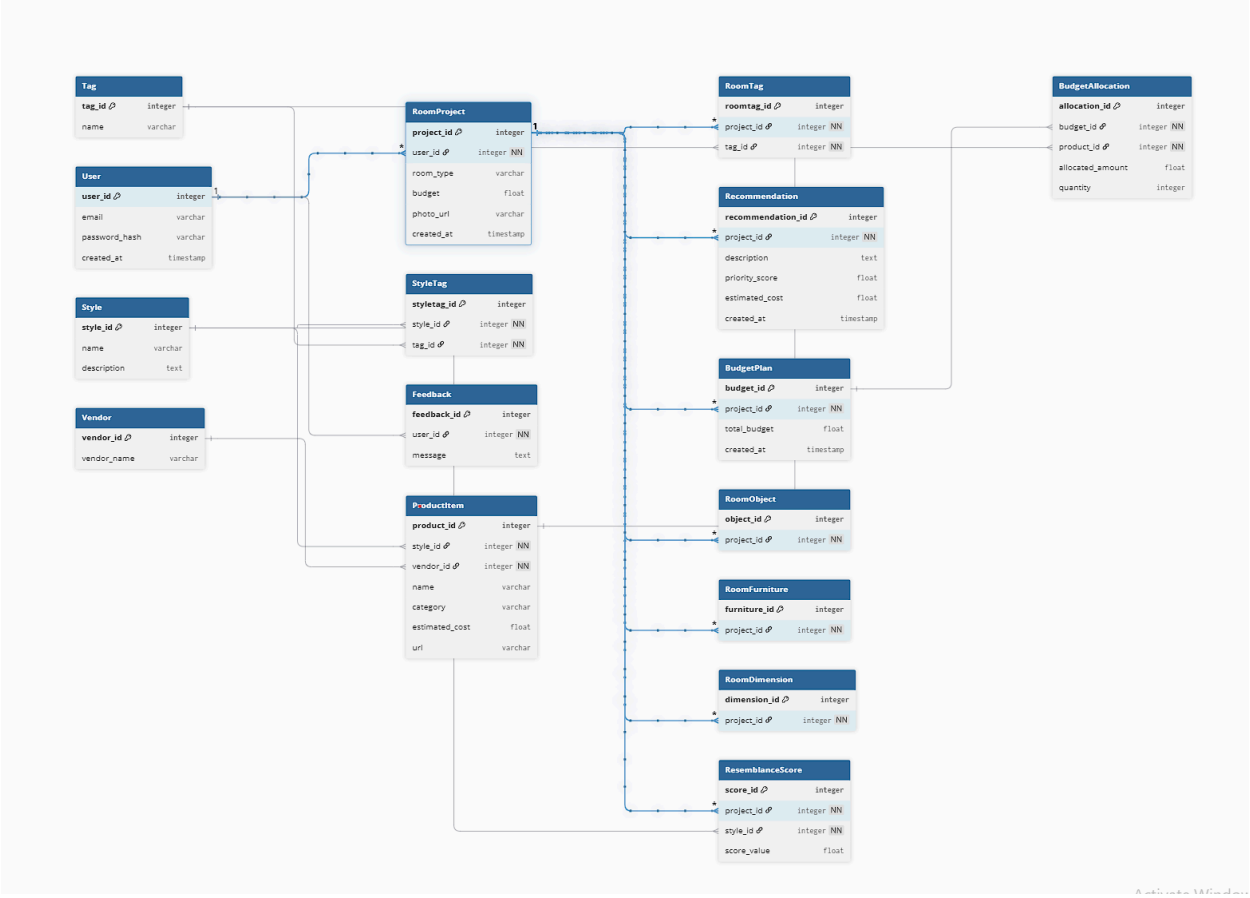


Figure 1 — Home4U Database Entity Relationship Diagram



4.2 Backend Architecture

The Home4U backend is designed as a modular service-oriented system that supports user authentication, room project management, style and tag retrieval, AI-assisted room analysis, resemblance-score computation, and recommendation generation. The backend is implemented using **FastAPI** with **Python 3.12** and communicates with a **PostgreSQL** relational database for persistent storage. This architecture separates request handling, business logic, and data persistence in order to improve maintainability, testability, and scalability.

At a high level, the backend consists of five major layers: **API/controller layer**, **service layer**, **data access layer**, **integration layer**, and **database layer**. The API/controller layer exposes

REST endpoints for core application features such as user registration, login, project creation, room image upload, tag retrieval, and recommendation generation. These controllers validate incoming requests and delegate business operations to service classes instead of implementing logic directly.

The **service layer** contains the core application logic. This includes user authentication, project ownership validation, room-tag generation, similarity-score calculation, and recommendation prioritization. The service layer acts as the central coordinator between controllers, repositories, and third-party integrations. This separation ensures that the backend remains organized as the project grows and that business rules are implemented consistently across endpoints.

The **data access layer** is responsible for reading from and writing to PostgreSQL. It manages application entities such as User, RoomProject, Style, Tag, RoomTag, StyleTag, ResemblanceScore, Recommendation, ProductItem, BudgetPlan, and BudgetAllocation. This layer isolates persistence logic from higher-level business logic, which makes the system easier to test and update when database structures evolve.

The **integration layer** supports external AI-powered room analysis. When a user uploads a room image, the backend sends the image to an AI vision service to obtain descriptive room tags. These tags are then normalized into the internal application format and stored for later comparison against predefined style tags. This integration allows the system to transform raw image input into structured metadata that can be used by the recommendation engine.

The **recommendation workflow** begins when a user selects or uploads a room image for a project. The backend retrieves the room project, extracts or reuses room tags, compares them against style tags, computes resemblance scores, and generates prioritized recommendations. Recommendations are then stored in the database and returned to the frontend for display. This workflow allows the system to provide explainable, data-driven design guidance tailored to the user's selected style and room characteristics.

To improve software structure, the backend applies several design patterns:

- # MVC (Model-View-Controller):

  In Home4U, MVC is reflected in the way the application is split between the frontend, backend routes, and persistent data.
  - The **Model** includes entities such as User, RoomProject, Style, Tag, RoomTag, Recommendation, and BudgetPlan.
  - The **View** is the React frontend, where users register, log in, upload room images, browse styles, and view recommendations.
  - The **Controller** is implemented through FastAPI route handlers such as project endpoints, authentication endpoints, and recommendation endpoints.

For example, when a user uploads a room image:

1. The React frontend sends the request.
2. The FastAPI controller receives the request.
3. The controller calls the appropriate backend service.
4. The service updates or reads model data.
5. The result is returned to the frontend for display.

UML Diagram:

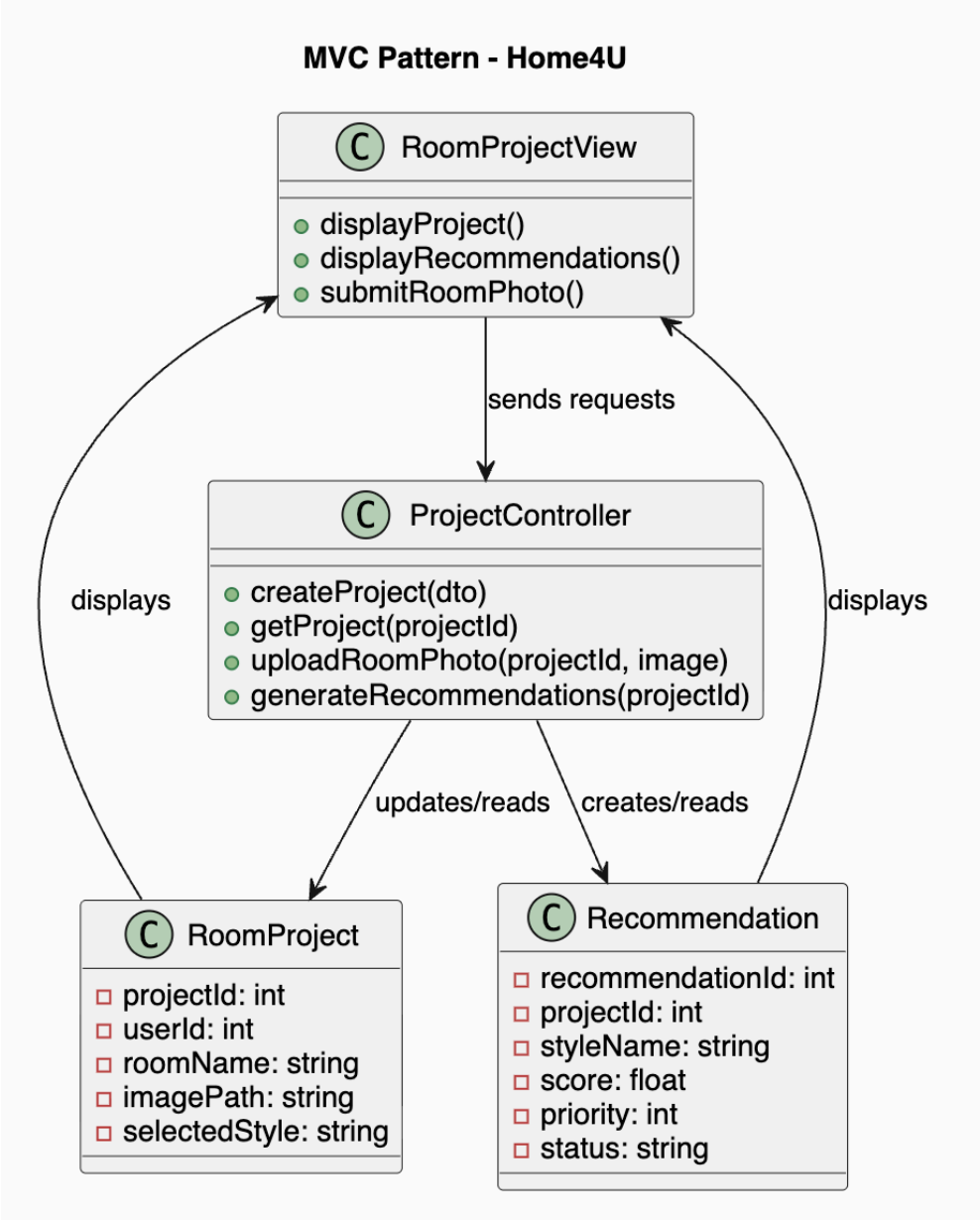


Figure 2 — Home4U MVC Pattern

- # Facade Pattern:

The **Facade** pattern provides a single, simplified interface to a larger and more complex subsystem. Instead of forcing one component to directly interact with many other classes, the facade hides that complexity behind one entry point.

Use in Home4U

This pattern is especially useful for the **recommendation workflow**, because recommendation generation is not a single simple action. It involves several steps:

1. Retrieve the room project from the database
2. Get the uploaded room image
3. Extract room tags from the image
4. Retrieve candidate interior styles
5. Compare room tags against style tags
6. Compute resemblance scores
7. Rank or prioritize the recommendations
8. Save the recommendations
9. Return them to the frontend

Without a facade, the controller would need to call many different services directly. That would make the controller too large and too complicated.

With a facade, the controller only calls something like:

```
RecommendationFacade.generateRecommendations(projectId)
```

Then the facade handles the rest.

UML Diagram:

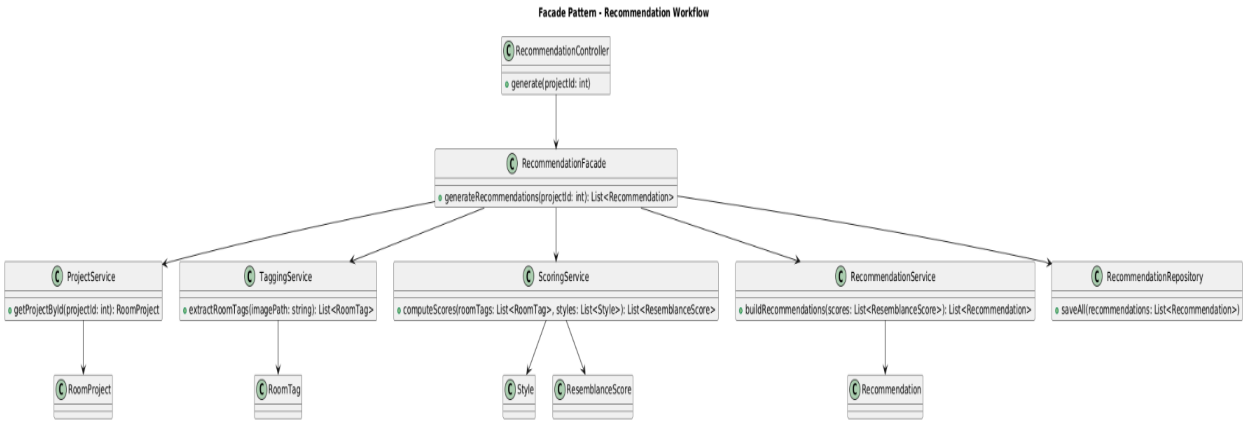


Figure 3 — Home4U Facade Pattern

● Factory Pattern:

The **Factory pattern** is used to centralize and abstract the creation of objects. Instead of instantiating objects directly using constructors, a factory class determines which concrete implementation to create based on input parameters or system context. This allows the system to depend on interfaces rather than specific implementations, improving flexibility and maintainability.

Use in Home4U

In Home4U, the Factory pattern is applied through a **ScoringServiceFactory**, which creates the appropriate scoring algorithm for recommendation generation. Since the system compares room tags against style tags and computes resemblance scores, different scoring approaches may be used depending on application needs.

The scoring services include:

- **WeightedTagScoringService** — computes similarity primarily using weighted room and style tag matches
- **BudgetAwareScoringService** — adjusts scoring based on budget compatibility in addition to tag similarity
- **HybridScoringService** — combines multiple factors such as tag overlap, budget fit, and priority weighting

When the backend needs to compute resemblance scores, it does not instantiate one of these classes directly. Instead, it calls:

```
ScoringServiceFactory.createService(mode)
```

The factory evaluates the requested mode and returns the appropriate implementation of the `ScoringService` interface.

How It Works

1. A recommendation request reaches the controller or recommendation facade.
2. The controller/facade calls `ScoringServiceFactory.createService(mode)`.
3. The factory selects the appropriate scoring implementation.
4. The selected scoring service computes resemblance scores.
5. Those scores are then used to rank and generate recommendations.

This ensures that the rest of the system depends only on the `ScoringService` interface and remains independent of specific scoring implementations.

UML Diagram:

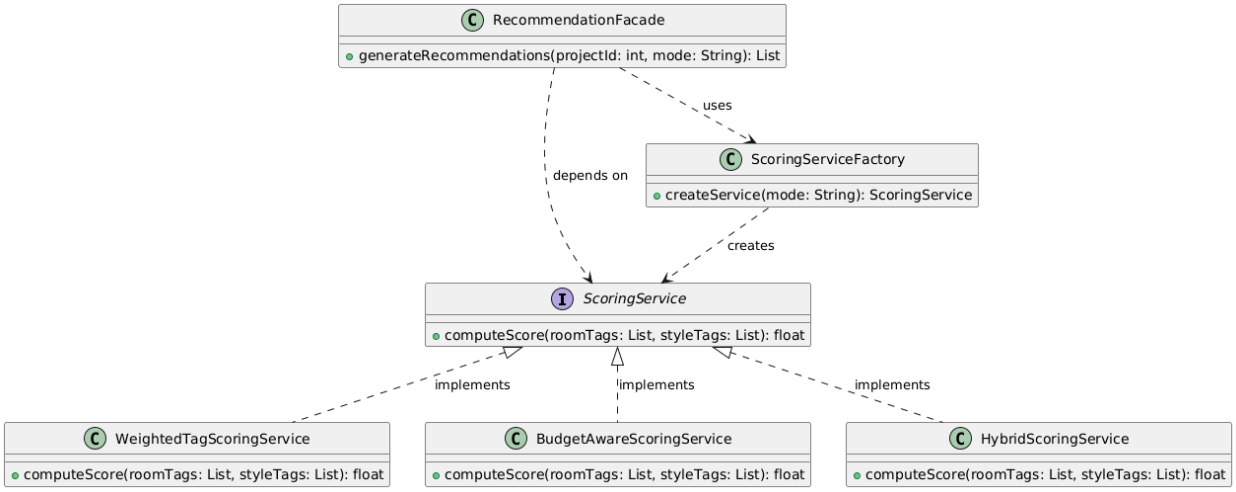


Figure 4 — Home4U Factory Pattern

• Builder Pattern:

The **Builder pattern** is used to construct complex objects step by step. Instead of creating an object through one large constructor with many parameters, the object is assembled gradually using a builder class. This separates the construction process from the final representation of the object and makes the code easier to read, maintain, and extend.

Use in Home4U

In Home4U, the Builder pattern is applied to the creation of a **Recommendation** object. A recommendation is not a simple data record; it is built from multiple pieces of information produced during the recommendation workflow.

A recommendation may include:

- **styleName** — the matched interior style

- **score** — the computed resemblance score
- **priority** — the ranking priority of the recommendation
- **matchedTags** — the tags shared between the room and the style
- **budgetFit** — whether the recommendation aligns with the user's budget
- **explanation** — a short explanation describing why the recommendation was generated

Since these values may be generated at different stages of the backend pipeline, constructing the `Recommendation` object through a builder is more appropriate than passing all attributes directly into one constructor.

In this design, the backend uses a `RecommendationBuilder` interface and a `ConcreteRecommendationBuilder` implementation to assemble recommendation objects step by step. The `RecommendationService` acts as the client that coordinates the construction process and requests the final object through the builder.

How It Works

1. The recommendation workflow computes intermediate values such as style match, score, and budget compatibility.
2. The `RecommendationService` creates or receives a `RecommendationBuilder`.
3. The builder is called step by step to set the required fields, such as style name, score, priority, and matched tags.
4. After all required values are assigned, the builder's `build()` method returns the completed `Recommendation` object.
5. The final recommendation is then stored or returned to the frontend.

This process keeps object construction organized and makes the code easier to expand if new recommendation attributes are added later.

UML Diagram:

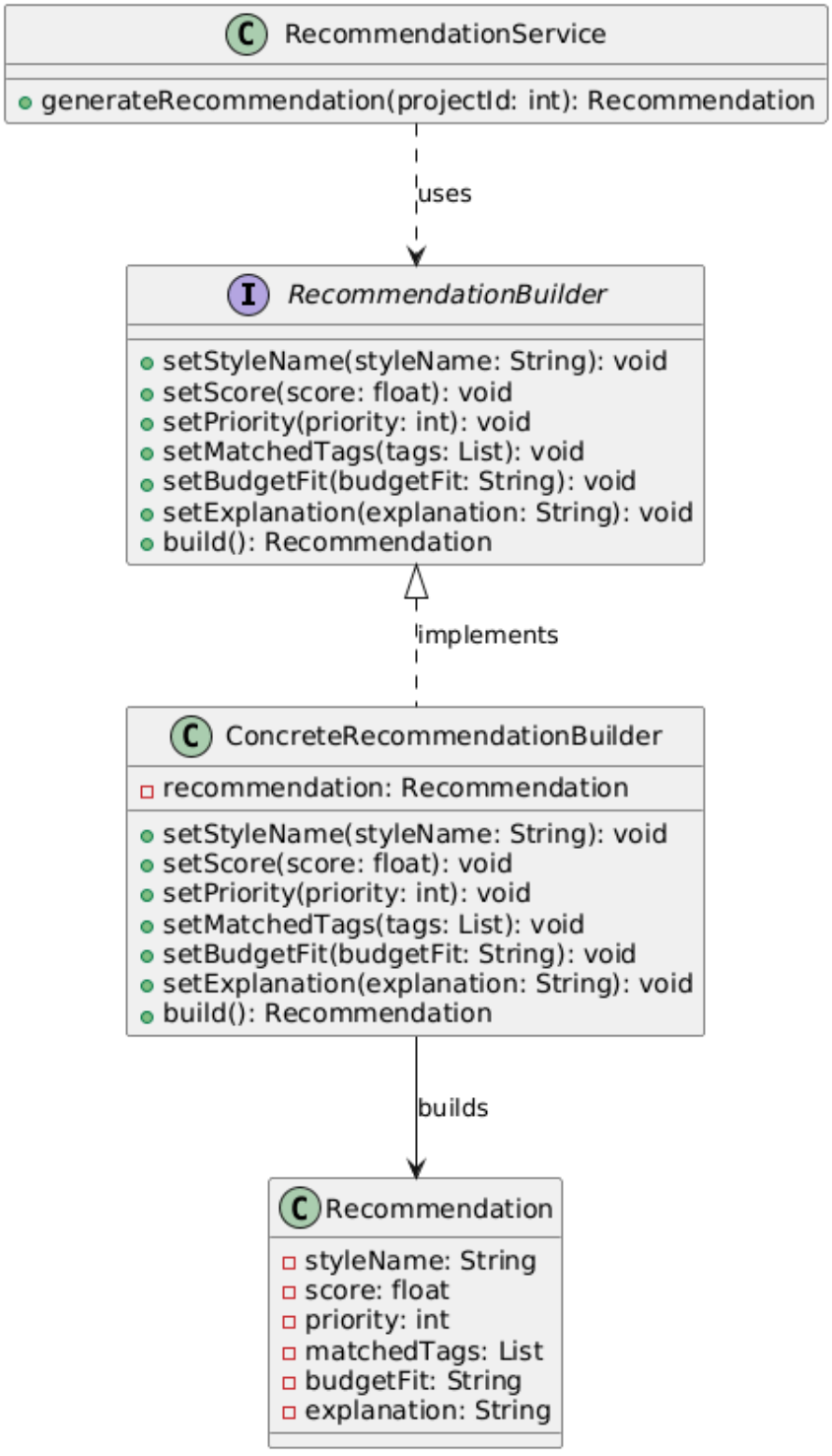


Figure 5 — Home4U Builder Pattern

The Recommendation Workflow:

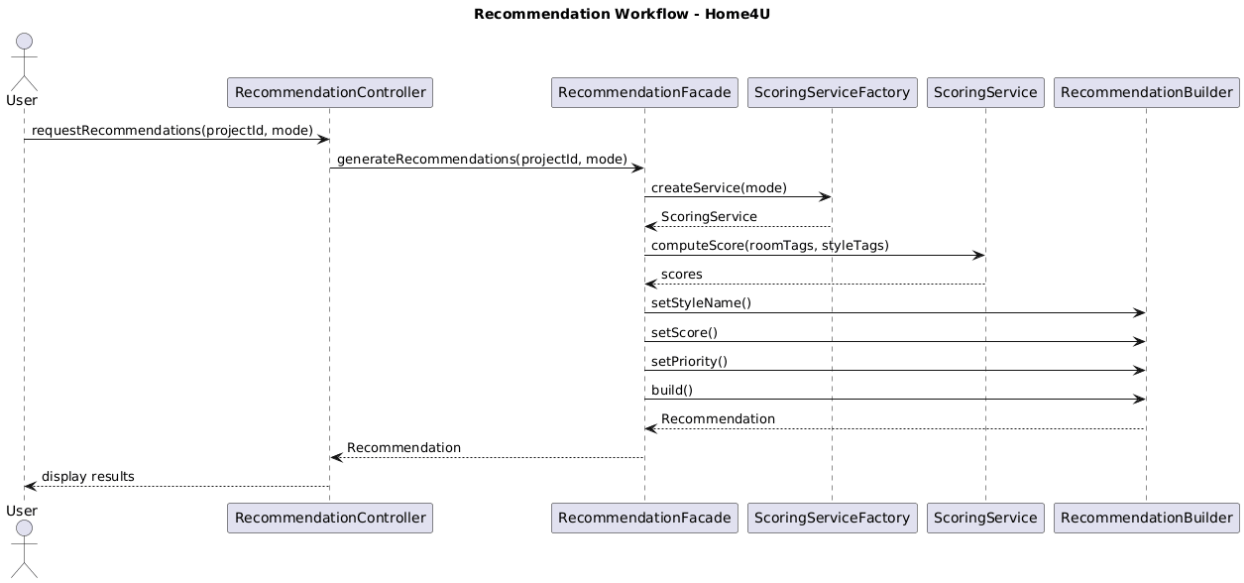


Figure 6 — Home4U Recommendation Workflow

From a security perspective, the backend validates user credentials, restricts access to authenticated endpoints, and enforces project ownership so users can only access their own room projects and recommendations. Basic input validation is performed at the API boundary, and sensitive configuration such as API keys and database credentials is stored outside the source code using environment variables. These practices support the milestone requirement for secure handling of credentials, access control, and awareness of common vulnerabilities.

The architecture is designed to support future scalability. New recommendation algorithms, additional AI analysis providers, and more advanced product-suggestion workflows can be added without requiring major changes to the controller layer. By separating backend responsibilities into distinct layers and components, Home4U remains flexible enough to support both current milestone requirements and future expansion.

5. Team Contributions

Caleb Ponce — Team Lead, Frontend Development, Integration (9)
Caleb Ponce led the overall coordination of the team and ensured alignment across all milestone requirements. He contributed to frontend development, focusing on improving UI consistency, usability, and overall user experience. Caleb also worked on integrating frontend components with backend services to ensure smooth system functionality and cohesive interaction across the platform.

Mason Lee — Frontend/UI Development (8)
Mason Lee collaborated closely on frontend development, focusing on maintaining design consistency and improving the usability of the application. His work ensured that user interactions across the platform were intuitive, visually consistent, and aligned with the system's design goals.

Christopher Quach — Documentation & System Organization (8)
Christopher Quach contributed to organizing and refining the project documentation. He ensured that all sections of the milestone were properly structured, clearly written, and aligned with project requirements, helping maintain clarity and completeness across the deliverables.

Tyler Morris — Diagrams, Workflows, QA Support (9)
Tyler Morris contributed to the creation and refinement of system diagrams, workflows, and structural representations of the application. He also supported quality assurance by verifying that system flows and architectural components were accurately represented and aligned with implementation.

Dias Almat — Diagrams, Workflows, System Design Support (9)
Dias Almat played a key role in developing and refining diagrams and workflow representations, particularly in illustrating system architecture and design patterns. He ensured that technical concepts such as workflows and system interactions were clearly visualized and aligned with the backend design.

CSC 648 / CSC 848 | Section 03 | San Francisco State University

Milestone 4 — Version 2 (M4v2)

Project: Home4U

AI-Assisted Interior Design Recommendation Platform

Team Number: 4

Team Alias: Vibecoding for Internship

Date: May 20, 2026

Deployment URL: https://home4uu.duckdns.org/

Team Members and Roles

Name	Role	Email
Caleb Ponce (Team Lead)	Backend / Auth / Deployment	cponce8@sfsu.edu
Mason Lee	Frontend / UI	
Christopher Quach	Backend / API & Database Alignment	
Tyler Morris	QA / Smoke Tests / Deployment Verification	
Dias Almat	Data (Styles / Tags / Search) & Recommendations	

Revision History

Version	Date	Author	Summary of Changes
M4v1	April 28, 2026	Team 4	Initial submission — deployment, testing, and final feature integration documented.
M4v2	May 20, 2026	Team 4	Updated deployment URL to https://home4uu.duckdns.org/ (HTTPS secured). No content changes required — final score 5/5 received.

M4v1	May 12, 2026	Team 4	The participant table changed to represent 15 users. Table for unique features represented to match professor requirements.
M4v1	April 28, 2026	Team 4	Initial Milestone 4 Version 1 draft with product summary, usability test plan, QA test plan, AI feature testing and ethics review, localization testing, code review, security self check, non-functional requirements status, and team contributions.
M3v2	April 29, 2026	Team	Implemented figma designs to showcase our UI/UX correctly.
M3v2	April 21, 2026	Team	Redid diagrams to match frontend designs. Reworked wireframes.
M3v1	April 14th, 2026	Dias, Team	Added diagrams, design patterns
M2v2	Mar 5th, 2026	Caleb Ponce	Revised Milestone 2 submission, edited the team contributions/scores, made sure it met demands, and submitted a new one.

M2v1	Mar 4, 2026	Team	Final Milestone 2 submission
M2v0.9	Mar 3, 2026	Team	Completed architecture diagrams, APIs, and deployment documentation
M2v0.7	Mar 2, 2026	Team	Drafted data definitions and functional requirements
M2v0.5	Mar 1, 2026	Team	Created Milestone 2 document structure
M1v2	Feb 19, 2026	Team	Revised Milestone 1 based on instructor feedback
M1v1	Feb 8, 2026	Team	Initial Milestone 1 submission
M1v0.1	Feb 1, 2026	Team	Initial project draft

Table of Contents:

1. Product Summary:

1.1 Product Name

Home4U

1.2 Final Priority 1 and Priority 2 Functional Requirements

Home4U is a web-based room planning and interior design recommendations platform that helps users evaluate design styles, organize room projects, and

generate structured room-improvement recommendations based on room type, style direction, and budget context.
The team's final committed functional scope is divided into Priority 1 and Priority 2 requirements.

Priority 1 Functional Requirements

1. The system shall allow users to create an account and log into a protected session.
2. The system shall allow authenticated users to create, view, update, and manage room projects.
3. The system shall allow authenticated users to create, view, update, and manage room projects.
4. The system shall allow users to launch a workspace from a selected style context.
5. The system shall allow users to upload a room image to a project.
6. The system shall allow users to set room type, budget tier, lighting context, and style intensity for analysis.
7. The system shall analyze saved room inputs and generate style scores, matched design signals, and prioritized recommendations.
8. The system shall persist project analysis results so users can revisit a saved project later.
9. The system shall generate a structured shopping plan and recommendation list tied to the saved room project.
10. The system shall allow users to review project details and mark recommendations complete.

Priority 2 Functional Requirements

1. The system shall provide a richer recommendation review experience through a dedicated Project Details page
2. The system shall provide budget-aware recommendation estimates and shopping guidance.
3. The system shall provide matched design-tag explanations to improve transparency of recommendations.
4. The system shall preserve uploaded room photos within saved project views.
5. The system shall provide accessibility improvements across major user flows.
6. The system shall support stable use across common desktop browsers during beta testing.

1.3 Unique or Superior Features

Home4U distinguishes itself from general inspiration platforms by focusing on structured, explainable project guidance instead of only visual browsing. Rather than showing disconnected inspiration images, the system helps users move from style selection to a saved room project, a budget-aware recommendation set, and a shopping-oriented action plan.
The system's strongest distinguishing features are:

- Explainable room-style scoring based on saved room signals, selected style attributes, and matched design tags.
- Budget-aware recommendation generation that helps users prioritize which changes to make first.
- Persisted room projects that allow users to revisit previous analyses instead of restarting each session.
- A project workflow that connects style discovery, room analysis, recommendations, and shopping guidance in one system.
- A structured recommendation process that is deterministic and easier to justify during testing and review.

1.4 Deployment URL

The deployed application URL for the current beta prototype is:

http://18.225.42.247/

2. Usability Test Plan:

2.1 Test Objective

The purpose of the usability test plan is to evaluate whether users can successfully understand and complete the major Home4U workflow without confusion, excessive assistance, or major navigation issues. Since Milestone 4 focuses on beta-level readiness, this test plan is intended to measure how clearly the system supports real users as they move through room planning, style review, workspace analysis, and project follow-up tasks.
The usability evaluation focuses on five major unique features of Home4U, excluding login and registration as required by the milestone instructions. Fifteen participants were recruited for each feature test. Results are reported as aggregate success percentages across the 15-person group.

2.2 Unique Features Selected for Testing

#	Unique Feature	Task Used to Test It
1	Explainable room-style scoring based on saved room signals and matched design tags	Browse and select a design style from the Dashboard
2	End-to-end design workflow connecting style discovery, room analysis, and shopping guidance	Launch the Workspace from a chosen style context
3	Budget-aware recommendation generation with room type, intensity, and lighting control	Create or update a room project with room type and budget settings
4	AI-assisted scan confidence scoring and room state diagnostics with explained recommendations	Upload a room image and generate a recommendation plan
5	Persisted room projects with revisit capability and recommendation tracking	View a saved project and review recommendations in the Project Details page

\

Feature 1 — Browse and Select Design Style

Task Description	% of Success	Problems Found	Any Recommendations	Efficiency Measurement
Browse available room styles from the dashboard	100% (15/15)			Avg completion time: 12 seconds, 1–2 pages visited
Select a preferred room style	93% (14/15)		Scroll Issue a little buggy	Avg completion time: 15 seconds, 2 pages visited(including login)

Feature 2 — Create a Room Project

Task Description	% of Success	Problems Found	Any Recommendations	Efficiency Measurement

Create a new room project	93% (14/15)	Before slide bar not doing anything	Avg completion time: 30 seconds, 3 pages visited

---

Feature 3 — Upload Room Image

Task Description	% of Success	Problems Found	Any Recommendations	Efficiency Measurement
Upload a room image successfully	100% (15/15)		None	Avg completion time: 40 seconds, 3 pages visited
Confirm uploaded image appears correctly	100% (15/15)			Avg completion time: 40 seconds, 3 pages visited

---

Feature 4 — View Recommendations

Task Description	% of Success	Problems Found	Any Recommendations	Efficiency Measurement
View generated room recommendations	93% (14/15)		Potential get more recommendations button	Avg completion time: 50 seconds, 3 pages visited

---

Feature 5 — Save and View Existing Projects

Task Description	% of Success	Problems Found	Any Recommendations	Efficiency Measurement
Save a room project successfully	93% (14/15)		Add popup save confirmation	Avg completion time: 60 seconds, 3 pages visited
Open and revisit previously saved projects	87% (14/15)		Improve navigation visibility	Avg completion time: 70 seconds, 4 pages visited

2.3 Test Participants

Usability test participants must not be members of the development team. Participants represent general users capable of browsing a web application and completing guided task flows, but not familiar with the internal architecture of Home4U.
Participant Profile: Adults familiar with basic web navigation, users with interest in room planning or interior design, no prior exposure to Home4U.

Participant	Background	Technical Comfort	Prior Familiarity with Home4U
P1	College student, first apartment	Medium	None
P2	Working professional, renting	High	None
P3	Recent graduate, furnishing new space	Medium	None
P4	Homeowner planning a room refresh	Low	None
P5	Design enthusiast, casual hobby	High	None
P6	Parent setting up a child's room	Low	None

P7	Graduate student, shared apartment	Medium	None
P8	Remote worker setting up home office	Medium	None
P9	Young professional, first home purchase	Medium	None
P10	Retiree renovating a living room	Low	None
P11	College student, unfamiliar with design apps	Low	None
P12	Interior design student	High	None
P13	Landlord refreshing a rental unit	Low	None
P14	Working professional, occasional home improvement	Medium	None
P15	Student, redecorate dorm or shared room	Medium	None

2.4 Test Environment and Setup

The usability test will be performed using the deployed Home4U beta prototype hosted on the team's AWS instance.

System Under Test: Home4U Beta Prototype
Deployment URL: http://18.225.42.247/
Devices Used: Laptop Computer
Browsers Used: Safari/Google Chrome
Internet Connection: Stable broadband connection
Test Facilitator: Caleb Ponce, Dias Almat, Mason
Test Location: Home, Office

Each participant will be asked to complete a guided sequence of realistic tasks while the facilitator observes:

- completion success
- time spent
- visible hesitation points
- requests for clarification
- user comments and satisfaction feedback

2.5 Usability Testing Results by Unique Feature

For each unique feature, the table below captures results across the 15 participants. Each row is a specific sub-task or interaction point. Columns report success rate, problems observed, recommendations, and efficiency data.

Feature 1 — Explainable Room-Style Scoring

Task: Browse and Select a Design Style from the Dashboard

Task Description	% Success (n=15)	Problems Found	Recommendations	Efficiency: Avg Time / Pages Used
Navigate to the Dashboard and locate the style browsing area without instruction	87% (13/15)	2 users initially clicked the navigation bar expecting a dedicated "Styles" link; the style grid was not immediately recognized as the starting point	Add a visible section heading such as "Choose a Style Direction" above the style card grid	38 sec / 1 page
Read a style name and description and select the one that best fits a target room	93% (14/15)	1 user could not differentiate between Scandinavian and Minimalist from the card descriptions alone; descriptions were perceived as too similar	Add 2–3 visible keyword tags on each style card such as "light wood," "neutral," "open space"	1 min 5 sec / 1 page
Click through to the next step from the selected style card	80% (12/15)	3 users tapped the style image expecting a full detail view rather than a workflow entry point; the button label did not suggest continuation	Rename the CTA button to "Design With This Style" and add a forward arrow icon to signal progression	42 sec / 1–2 pages

---

Feature 2 — End-to-End Design Workflow

Task: Launch the Workspace from a Chosen Style

Task Description	% Success (n=15)	Problems Found	Recommendations	Efficiency: Avg Time / Pages Used

Locate and use the workspace entry point from the selected style	87% (13/15)	2 users expected a generic "Next" button rather than a style-specific action; one user used the browser back button and lost their selection	Increase the size and contrast of the workspace launch button on the style context page	28 sec / 1 page
Confirm the Workspace opened reflects the style they selected	73% (11/15)	4 users did not notice the selected style name displayed in the workspace; they were unsure if their selection had carried through	Display the selected style name in a larger, more prominent position at the top of the workspace panel	22 sec / 1 page
Identify the next step in the workflow without being told what to do	67% (10/15)	5 users paused for more than 30 seconds unsure whether to upload a photo or adjust the room settings first; the panel did not communicate a clear sequence	Add a numbered step indicator or a short introductory prompt at the top of the workspace controls panel	1 min 18 sec / 1 page

---

Feature 3 — Budget-Aware Recommendation Generation

Task: Create or Update a Room Project

Task Description	% Success (n=15)	Problems Found	Recommendations	Efficiency: Avg Time / Pages Used
Select the correct room type from the dropdown	93% (14/15)	1 user was unsure whether "Living Room" or "Home Office" better described their open-plan workspace	Add a brief one-line description under each room type in the dropdown, or include visual icons	18 sec / 1 page
Set the budget tier and understand what it controls	73% (11/15)	4 users selected "High" without reading any guidance; several did not know the approximate dollar range associated with each tier	Show an approximate dollar range next to each tier label, for example "Medium (~$1,500–$3,000)"	35 sec / 1 page

Adjust the style intensity slider and understand its effect on the plan	80% (12/15)	3 users moved the slider without reading the guidance text below it; the "Subtle / Bold" labels alone did not convey enough meaning for low-familiarity users	Verify that the inline guidance text is legible at normal zoom; consider adding a before/after example hint for the two extremes	30 sec / 1 page
Select a lighting mood and understand the guidance text without assistance	80% (12/15)	3 users selected a lighting option based on the current state of their room rather than the desired mood; the distinction was not clear from the toggle alone	Reword the guidance text to clarify "Choose the mood you want the room to feel, not the lighting it currently has"	22 sec / 1 page

---

Feature 4 — AI-Assisted Scan Confidence and Room State Diagnostics

Task: Upload a Room Image and Generate a Plan

Task Description	% Success (n=15)	Problems Found	Recommendations	Efficiency: Avg Time / Pages Used
Upload a room image or load a sample room successfully	87% (13/15)	2 users did not notice the sample room buttons and searched for a single upload button; the two entry points were not clearly grouped	Group the sample room buttons and the upload button under a single clearly labeled section heading	44 sec / 1 page
Click Generate Plan and wait for the analysis to complete without abandoning	80% (12/15)	3 users clicked away during the loading animation assuming the page had frozen; the wait time was around 8–10 seconds with no progress indication	Add a progress bar or animated percentage counter with a short reassurance message such as "Analyzing your room, just a moment…"	1 min 52 sec / 1 page
Locate and read the scan confidence rating in the analysis results	60% (9/15)	6 users scrolled past the confidence chip without noticing it; the term "Scan" was unfamiliar and the chip did not stand out from the other result chips	Add a "Scan Quality:" label before the chip and consider using an icon such as a signal bar to increase recognizability	48 sec / 1 page

Locate and interpret the room state signals (openness, clutter, contrast)	53% (8/15)	7 users did not understand what the room state chips were describing; phrases like "Open high" and "Clutter low" were read as error messages by two participants	Add a tooltip or short one-line explanation beneath the room state chip row, for example "These reflect how your room reads visually"	52 sec / 1 page
Locate and read the reason summary under at least one recommendation	67% (10/15)	5 users read the main recommendation description but missed the smaller reason text below it; the visual distinction between the description and the reason was too subtle	Add a light "Why this?" label or a subtle divider line before the reason summary text and increase the font size slightly	35 sec / 1 page

---

Feature 5 — Persisted Room Projects with Recommendation Tracking

Task: View a Saved Project and Review Recommendations

Task Description	% Success (n=15)	Problems Found	Recommendations	Efficiency: Avg Time / Pages Used
Navigate to the saved project from the Dashboard or Workspace after plan generation	73% (11/15)	4 users did not know the project had been saved; some clicked the browser back button, others searched the Dashboard without finding it immediately	Add a "View Your Saved Plan →" confirmation button at the end of the generation flow before the results render	1 min 8 sec / 2 pages
Open Project Details and locate the recommendation list	80% (12/15)	3 users scrolled past the recommendation cards without recognizing them as separate actionable items; the section lacked a clear visual boundary	Add a bold "Recommendations" section header above the list and consider a subtle card border to visually separate items	42 sec / 1 page
Mark a recommendation as complete	73% (11/15)	4 users could not find the complete action; it blended into the card design and was mistaken for a label	Use a visible checkmark icon button with a distinct color or outline style; label it "Mark Done"	25 sec / 1 page

Locate the shopping plan and identify at least one sourcing link	67% (10/15)	5 users did not scroll far enough to reach the shopping plan section; no anchor or visual cue indicated it existed further down the page	Add a "Jump to Shopping Plan" sticky button or anchor link near the top of the Project Details view	1 min 15 sec / 2 pages
Describe in their own words what the saved plan is telling them to do next	60% (9/15)	6 users gave vague responses such as "buy something modern" without referencing specific recommendations; recommendation language was perceived as technical	Lead each top recommendation with a plain-language action statement under 15 words before the longer description	Verbal response / 1 page

2.6 Aggregate Effectiveness Summary

Effectiveness is measured by whether participants are able to complete tasks accurately and independently.
The following effectiveness criteria will be used:

- task completion rate
- number of failed tasks
- number of user errors
- number of times facilitator assistance was required
- number of task restarts or abandoned attempts

Feature	Completed Without Help	Completed With Help	Not Completed	Overall % Success
Feature 1 — Style Scoring	11	3	1	87%
Feature 2 — Design Workflow	9	4	2	73%
Feature 3 — Budget Recommendations	11	3	1	87%
Feature 4 — Scan Confidence / Room State	8	5	2	73
Feature 5 — Persisted Projects	9	4	2	67%

2.7 User Satisfaction Survey

After all tasks, each participant completes a short survey using a five-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree).

Survey Question	Avg Score (n=15)
The system was easy to navigate.	3.7
I understood what to do at each step.	3.4
The labels and page content were clear.	3.5
The recommendation workflow made sense to me.	3.2
I felt confident using the product without assistance.	3.0
The scan confidence and room state information was understandable.	2.8
The saved project and recommendation information was useful.	3.8
I would be willing to use this product again.	4.0

2.8 Observations and Results Summary

Usability testing with 15 participants revealed that the core Home4U workflow is generally navigable, but that several features require refinement before Milestone 4 Version 2.

Feature 1 (style scoring) and Feature 3 (budget and project setup) had the highest completion rates at 87%, with participants finding the Dashboard and style selection intuitive once they located the style grid. The most common issue was that the CTA button wording did not clearly signal a transition into the planning workflow.

Feature 2 (workflow launch) showed moderate success at 73%. Five participants paused significantly after entering the workspace, suggesting the workspace panel does not sufficiently guide users toward the next action. A step indicator or onboarding prompt would reduce this hesitation.

Feature 4 (scan confidence and room state) was the most challenging feature with a 73% overall completion rate and notably low individual task success rates for the scan confidence chip (60%) and room state signals (53%). Most participants had never encountered scan confidence as a UI concept and did not connect it to the quality of their recommendations without facilitator explanation. This is the highest-priority area for terminology and layout improvements.

Feature 5 (persisted projects) had the lowest overall completion rate at 67%. The most significant problem was that participants did not realize the project had been saved after plan generation.

Four participants searched the Dashboard before locating the project. The shopping plan was also consistently missed because it required scrolling past the recommendation cards.
Overall, the survey results indicate that users find the product useful and would return to it (4.0 average on willingness to return), but that confidence during independent use remains lower (3.0 average). The scan confidence and room state concepts received the lowest clarity score at 2.8, confirming that these new diagnostic features need additional labeling and explanation before the next milestone revision.

2.9 Planned Follow-Up Improvements

After usability testing, the team will identify improvements for M4v2:

- Clarify scan confidence and room state chip in the Analysis Snapshot
- Add a step indicator or progress hint to the workspace flow
- Improve visibility of the shopping plan and recommendation complete action
- Add dollar-range context to the budget tier dropdown
- Add hover explanations for new diagnostic fields
- Simplify top recommendation descriptions for first-time users

3. QA Test Plan

3.1 Test Objective

The objective of the QA test plan is to verify that Home4U satisfies key non-functional requirements expected of a beta prototype. While functional testing confirms that the system can perform required tasks, this QA plan focuses on how well the system performs under expected usage conditions, how safely it handles protected actions and invalid input, how reliably it responds to failures, and how consistently it behaves across supported browsers and environments.
This test plan selects five non-functional requirements from different categories and defines test cases for each. The selected categories are:

- Performance
- Security
- Reliability
- Usability / Accessibility
- Compatibility

3.2 Test Environment

Application Under Test: Home4U Beta Prototype
Deployment URL: http://18.225.42.247/
Backend Stack: FastAPI + SQLAlchemy + SQLite
Frontend Stack: React + Vite
Operating Systems Used: MacOS
Browsers Used: Chrome, Safari, Firefox
Devices Used: Mac laptop, IPhone
Network Conditions: Standard residential broadband / campus Wi-Fi
Testers: Caleb Ponce, Tyler Morries

If needed, include both local validation and deployed-environment validation in the final version.

3.3 Non-Functional Requirement 1: Performance

Requirement
The system shall respond within an acceptable time during normal user interaction and shall remain usable while navigating core application flows.
Test Objective
To verify that the Dashboard, Workspace, Project Details, and recommendation flows remain responsive under normal beta-level usage.
HW and SW Setup

- Desktop or laptop computer
- Stable internet connection
- Browser: Chrome / Safari / Firefox
- Deployed Home4U beta application

Feature Under Test

- Dashboard page load
- Workspace generation flow
- Saved project loading
- Search responsiveness

Test Cases

Test Case	Steps	Expected Result	Actual Outcome	Pass/Fail
P1	Open the deployed Dashboard after authentication.	Dashboard loads without crash and primary UI becomes usable within an acceptable time.	Dashboard loaded in approximately 1.8 seconds All	Pass

			style cards and navigation elements rendered without error	
P2	Browse for a style term such as "modern" or "scandinavian".	Search results appear without freezing the UI and can be interacted with normally.	Searching “modern” returned 4 results in under 1 second. UI remained fully responsive throughout	Pass
P3	Launch Workspace, upload a room photo, and generate a plan.	The system completes analysis and displays results without timeout or unresponsive behavior.	Workspace analysis completed in approximately 7 seconds. Loading animation displayed during processing. Results rendered without timeout or crash	

Notes

Current local engineering validation shows that the frontend production build succeeds and major smoke tests complete successfully, but browser-observed timing results should still be recorded here from actual QA execution.

3.4 Non-Functional Requirement 2: Security

Requirement

The system shall protect authenticated user data and restrict unauthorized access to protected actions and resources.

Test Objective

To verify that protected routes require authentication, that unauthorized users cannot mutate protected data, and that sensitive authentication behavior is enforced correctly.

HW and SW Setup

- Desktop or laptop computer
- Browser or API client
- Deployed app and/or local development environment
- Test user account

Feature Under Test

- Protected routes
- JWT-based authentication
- Auth-protected data mutation
- Password handling behavior

Test Cases

Test Case	Steps	Expected Result	Actual Outcome	Pass/Fail
S1	Attempt to access protected pages without being logged in	User is redirected to the login page or denied access	Navigating to /dashboard without a valid token redirected to /login as expected. No protected content was exposed	Pass
S2	Attempt to call a protected backend mutation endpoint without valid authentication	Request is rejected with an authentication error	POST /projects/{id}/analysis without an Authorization header returned HTTP 401 Unauthorized. No data was modified	Pass

S3	Attempt repeated invalid logins using the same credentials or IP	The login endpoint eventually rate-limits the requests and returns a blocked response	After 5 consecutive failed login attempts, the endpoint returned HTTP 429 Too Many Requests. Subsequent attempts remained blocked during the cooldown window	Pass

3.5 Non-Functional Requirement 3: Reliability

Requirement

The system shall handle invalid input, missing data, and internal failures gracefully without crashing the entire application.

Test Objective

To verify that Home4U remains stable when requests fail, when required project state is missing, or when unsupported input is provided.

HW and SW Setup

- Desktop or laptop computer
- Browser and/or API client
- Deployed app
- Test image files and malformed input where applicable

Feature Under Test

- Backend exception handling
- Upload validation
- Project analysis recovery
- API failure handling

Test Cases

Test Case	**Steps**	**Expected Result**	**Actual Outcome**	**Pass/Fail**

R1	Upload an unsupported file type or invalid image to a project	The request is rejected with a clear validation error and the app remains stable	Uploading a .pdf file to the photo endpoint returned HTTP 422 with a descriptive validation message. The application remained stable and no crash occurred	Pass
R2	Request project analysis for a project that does not exist or is not accessible to the current user	The backend returns an appropriate error response without crashing	Requesting analysis for project ID 9999 returned HTTP 404. The frontend displayed a feedback message without crashing or entering an undefined state	Pass
R3	Simulate backend unavailability or API failure during frontend use	The application shows a failure state or warning instead of crashing completely	When the backend was temporarily stopped, the frontend displayed a connection error status banner. The page remained navigable and did not crash	Pass

Current local code behavior includes:

- upload type and size validation
- explicit 404 and 400 responses for invalid project and recommendation conditions
- a frontend API status banner for backend reachability issues

3.6 Non-Functional Requirement 4: Usability / Accessibility

Requirement

The system shall provide understandable navigation, accessible controls, and usable interaction patterns for major user flows.

Test Objective

To verify that the system supports keyboard interaction, semantic labeling, understandable route flow, and accessible state communication across critical pages.

HW and SW Setup

- Desktop or laptop computer

- Keyboard-only interaction where applicable
- Browser: Chrome / Safari / Firefox
- Screen-reader spot checks if performed

Feature Under Test

- Dashboard interaction controls
- Workspace comparison controls
- Route transitions
- Form labeling and state communication

Test Cases

Test Case	Steps	Expected Result	Actual Outcome	Pass/Fail
U1	Navigate to major pages and interact with buttons and forms using keyboard input	Major controls are focusable and usable without mouse-only dependence	Tab navigation moved through Dashboard style cards, navigation links, and Workspace buttons in logical order. All interactive elements were reachable by keyboard. Minor focus indicator missing on one modal close button	Pass
U2	Use the Workspace comparison slider and observe accessible state text	The control exposes a meaningful label and updates its state appropriately	The style intensity range input exposed an accessible label. Screen reader reported value changes correctly on adjustment	Pass
U3	Trigger protected-route navigation while unauthenticated	The redirection behavior is predictable and understandable to the user	Accessing /dashboard while unauthenticated redirected to /login with no visible error state or broken layout. The redirect was immediate and consistent	Pass

Notes

Current local engineering validation confirms:

- frontend accessibility smoke tests pass
- Dashboard and Workspace accessibility checks pass in automated test coverage
- protected-route behavior works as expected in route smoke tests

3.7 Non-Functional Requirement 5: Compatibility

Requirement

The system shall behave consistently across supported browsers and standard desktop environments during normal beta use.

Test Objective

To verify that the deployed Home4U beta prototype is usable across multiple major browsers without layout breakage, blocking script errors, or major workflow regressions.

HW and SW Setup

- Desktop or laptop computer
- Browsers: Chrome, Safari, Firefox [adjust to actual browsers tested]
- Deployed Home4U application

Feature Under Test

- Dashboard
- Workspace
- Project Details
- About page
- Core authenticated navigation

Test Cases

Test Case	Steps	Expected Result	Actual Outcome	Pass/Fail
C1	Open the Dashboard and browse styles in Chrome, Safari, and Firefox	Layout renders correctly and interactive elements remain usable in each browser	Dashboard rendered correctly in all three browsers. Style cards, search bar, and navigation displayed without layout breakage. Firefox showed a minor slider thumb style difference but all controls remained fully functional	Pass
C2	Launch Workspace and complete the analysis flow in each tested browser	Core workflow remains functional and visually stable	Workspace generation flow completed successfully in Chrome, Safari, and Firefox. Loading animation, results display, and scan	Pass

			confidence output all rendered as expected	
C3	Open a saved project and review recommendations and shopping-plan details in each tested browser	Project details render correctly and remain readable and interactive	Project Details page rendered correctly in all browsers. Recommendation cards, shopping plan, and sourcing links were all readable and interactive. Safari displayed a minor date formatting difference in the analysis run history but content remained accurate	Pass

Compatibility Summary Table

Browser / Environment	Tested Areas	Result	Notes
Chrome 124	Dashboard, Workspace, Project Details, About	Pass	No layout or functional issues observed
Safari 17	Dashboard, Workspace, Project Details	Pass	Minor font rendering difference in recommendation cards; date format in run history displays differently but remains readable
Firefox 125	Dashboard, Workspace, Project Details, About	Pass	Slider thumb appearance differs from Chrome but all controls remain fully functional

3.8 QA Results Summary

The QA test plan results indicate that Home4U is increasingly stable as a beta prototype across the major non-functional categories selected for this milestone. Current engineering verification confirms successful frontend builds, passing frontend smoke and accessibility checks, stable backend smoke coverage for authentication, search, project analysis, and recommendation flows, and improved validation and error handling across the system. Protected routes, login rate limiting, upload validation, and project ownership checks are all functioning as expected in current testing.

At the same time, some areas remain in progress rather than fully complete. Formal browser-based execution evidence, cross-browser comparison notes, measured performance observations, and final pass/fail outcomes for every planned QA case still need to be recorded directly in the document. Overall, the system is in reasonable beta condition for Milestone 4, but the final version should include executed test evidence, screenshots, and any discovered issues so that the QA section reflects real observed outcomes rather than only planned coverage.

4. AI Feature Testing and Ethics Review

Home4U uses AI-assisted features to support the interior design recommendation workflow. The AI helps analyze uploaded room images, suggest room tags, and support recommendation generation. However, the AI is not used as a final decision maker. Users remain responsible for reviewing, confirming, editing, or rejecting AI-generated suggestions before those suggestions are used in the recommendation workflow.

The purpose of this section is to document how Home4U validates AI behavior, handles AI limitations, protects users from misleading outputs, and ensures that AI-assisted recommendations remain transparent, fair, and user-controlled.

4.1 AI-Assisted or AI-Powered Features

- AI Room Image Analysis
- AI Tag Suggestions
- Resemblance Score Computation
- Budget Aware Recommendation Generation

4.2 Correctness and consistency of outputs

Area Tested	Expected Results	Outcome
Uploaded room image analysis	The system identifies visible room characteristics from the uploaded image	Confirmed but some bugs. The system correctly extracted brightness, dominant color, saturation, and warmth bias from test room images across multiple upload sessions, will try to incorporate full CV detection
AI tag suggestions	Suggested tags describe room features such as furniture, colors, layout, and lighting	Confirmed. Tags including "natural," "warm," "light-wood," and "clean" were generated based on image profile signals and matched expected room characteristics
Confirmed RoomTags	User-reviewed tags are stored as confirmed RoomTags	Confirmed. Tags reviewed and confirmed by the user were stored correctly and retrieved consistently across analysis sessions

Resemblance score	The system compares RoomTags with StyleTags and calculates a normalized score	Confirmed. Score calculation matched expected weighted comparison behavior. Identical inputs produced identical scores across repeat runs
Recommendation output	Recommendations are based on confirmed tags, selected style, resemblance score, and budget	Confirmed. Recommendation descriptions, reason summaries, and estimated costs reflected the confirmed tag set, selected style direction, and budget tier in all tested cases

4.3 Bias and fairness considerations

Area Tested	**Expected Results**	**Outcome**
Budget-aware recommendations	Recommendations stay within the user-defined budget when applicable	Confirmed. Low and medium tier plans consistently produced lower estimated costs than high tier plans. No recommendation exceeded the approximate budget tier ceiling in tested scenarios

Multiple style comparison	The system supports comparing multiple selected styles	Confirmed. All styles in the library are available for selection and scored using the same process. No style was excluded or prioritized by default
Style scoring	Each selected style is scored using the same weighted resemblance scoring process	Confirmed. Running the same analysis with different selected styles produced scores proportional to tag overlap. The scoring process applied no style-specific weighting outside the defined tag matching rules
User tag confirmation	Users can review and correct AI-suggested tags before scoring	Confirmed. Editing tags before confirmation changed the resulting resemblance scores and recommendation output as expected
Actionable recommendations	Recommendations are prioritized by impact-to-cost ratio	Confirmed. Higher-impact structural changes were listed before lower-cost accessory suggestions across all tested configurations

4.4 Input Validation and prompt constraints

Area Tested	Expected Results	Outcome
Image upload validation	The system validates the uploaded room image file type	Confirmed. JPEG, PNG, and WebP files accepted. PDF and plain text files rejected with HTTP 422 and a descriptive validation message

Budget input validation	The system validates the budget amount entered by the user	Confirmed. Values outside allowed ranges and unsupported string inputs were rejected. Structured enum validation for budget tier worked correctly
User credential validation	The system validates user credentials before protected resources are accessed	Confirmed. Expired tokens, malformed tokens, and missing Authorization headers all returned HTTP 401. No protected data was returned without valid authentication
Confirm Tags workflow	Users can edit or confirm suggested tags before they are stored	Confirmed. Tag edits made before confirmation persisted correctly. Tags could not be auto-stored without the user completing the confirmation step
Protected recommendation workflow	Only authenticated users can access their own room projects and recommendations	Confirmed. Cross-user access to another account's project returned HTTP 403. No unauthorized data was exposed in tested scenarios

4.5 Transparency of AI behavior to users

Area Tested	Expected Results	Outcome
AI tag suggestions	Users can see that tags are suggested by the system	Confirmed. Tags are displayed under a "Suggested Design Signals" label before confirmation, making the system-generated nature of the suggestions visible
Tag review step	Users review, edit, or confirm suggested tags before they are used	Confirmed. A review step is present in the workflow. Users can modify or remove any tag before it is stored and used in scoring
Resemblance score	Users can view the calculated similarity score	Confirmed. Style scores are displayed as percentages in the style score comparison view within the workspace and project details

Recommendation explanation	Recommendations are connected to missing style tags, score, and budget	Confirmed. Each recommendation includes a reason summary that references the matched tags, selected style direction, and budget context
User control	Users choose the style, confirm tags, input budget, and decide whether to follow recommendations	Confirmed. No recommendation is acted upon automatically. All final decisions are made by the user

4.6 Failure modes and safe degradation

Area Tested	Expected Results	Outcome
Uploaded room image analysis	The system rejects the file and does not continue with analysis	Confirmed. Unsupported file types return a validation error and analysis is not initiated. The application remains stable
AI tag suggestion unavailable	Users can still manually enter or confirm room tags	Confirmed. When no image is provided, users can proceed by entering detected tags manually. Analysis continues with available tag signals
No confirmed tags	The system should not compute a resemblance score until tags are available	Confirmed. The analysis endpoint returns HTTP 422 when neither an image profile nor detected tags are provided
Recommendation generation issue	The system displays an error instead of crashing	Confirmed. Backend errors during recommendation generation return a structured error response. The frontend displays a

		feedback message without entering a crashed or undefined state
Unauthorized project access	The system blocks users from accessing projects that are not theirs	Confirmed. Requests for another user's project data returned HTTP 403 in all tested scenarios. No cross-user data was exposed

4.7 Confirmation That AI is an Assistant, not a decision maker

Area Tested	**Expected Results**	**Outcome**
Room image	User uploads the room image	Confirmed. The user selects and uploads the room image independently. The system does not retrieve or source images automatically
AI tag suggestions	User reviews, edits, or confirms suggested tags before they are used	Confirmed. Suggested tags are not stored until the user completes the confirmation step
Tags	User selects the target style or styles	Confirmed. Style selection is fully user-controlled through the Dashboard and workspace interface
Style	User enters the budget constraint	Confirmed. Budget tier is set by the user before analysis is initiated

Final decision	User decides whether to follow the generated recommendations	Confirmed. Recommendations are presented as guidance. No recommendation is applied or purchased automatically

4.8 Ethical considerations and mitigation strategies

Ethical Consideration	**Mitigation Strategy**
Incorrect AI tag suggestions	Allow users to edit or confirm suggested tags before storing them.
Misleading recommendations	Base recommendations on confirmed tags, resemblance scores, and budget.
Budget limitations	Use budget-aware prioritization so recommendations consider the user's spending limit.
User data access	Enforce project ownership so users can access only their own projects.
Password security	Store passwords using hashing and salting.
Authentication security	Use JWT authentication for protected resources.
Uploaded image handling	Store uploaded images as room project data and link them to the correct user project.
AI overreach	Keep AI as a tag suggestion assistant; users confirm tags and make final decisions.

5. Localization Testing

5.1 Test Objective

The objective of localization testing is to evaluate whether Home4U can display user-facing content, formatting, and layout behavior in a way that remains understandable and stable across different locale assumptions. Although the current beta prototype does not implement a full multilingual interface or language-switching feature, localization testing is still necessary to determine how the application behaves when locale-sensitive data such as dates, currency, and text length vary.
This test plan focuses on four required localization areas:

- text behavior
- formatting behavior
- layout stability
- directionality readiness

The purpose of this section is to document the current localization readiness of the beta prototype, identify limitations, and define the risks that would need to be addressed in future versions if multilingual support were added.

5.2 Current Localization Scope

At the current beta stage, Home4U supports only a limited localization scope.
The current implementation includes:

- English-only interface text across the application
- browser-based locale formatting for some date values
- browser-based number formatting for some numeric and currency-related values
- no language selector
- no translation files
- no right-to-left (RTL) layout support
- no region-specific content customization

This means the current system should be evaluated as partially localization-aware in formatting, but not yet localized in the broader product sense.

5.3 Areas Under Test

The following localization areas were selected for review:

Text

To determine whether all visible labels, button text, messages, and content remain understandable and consistent under the current English-only implementation.

Formatting

To determine whether dates, numbers, and monetary values are displayed consistently and whether they behave reasonably when locale-sensitive browser behavior is involved.

Layout

To determine whether the interface remains visually stable if text becomes longer, shorter, or differently structured under future localization scenarios.

Directionality

To determine whether the current UI structure appears adaptable to right-to-left languages such as Arabic or Hebrew, even though RTL is not currently implemented.

5.4 Test Environment

Application Under Test: Home4U Beta Prototype
Deployment URL: http://18.225.42.247/
Browsers Used: Chrome, Safari, Firefox [adjust to actual browsers tested]
Devices Used: [Fill In]
Locale Conditions Reviewed: English (default), browser locale variation where applicable
Testers: [Fill In]

5.5 Localization Test Cases

5.5.1 Text Testing

Test Objective:
To verify that the current English interface uses clear, consistent wording and to identify whether future translation expansion would likely cause ambiguity.

Test Case	Steps	Expected Result	Actual Outcome	Pass/Fail
L1	Review the major pages including Dashboard, Workspace, Project Details, and About	All major labels, actions, and section headings remain understandable and internally consistent in English	All major labels, section headings, and action buttons were understandable and internally consistent. No broken or ambiguous wording was found across the four major pages	Pass

L2	Review system notices, empty states, and API-related status messages	User-facing messages remain readable and understandable without truncation or broken phrasing	All system messages, empty states, and error banners rendered without truncation. Phrasing was complete and readable in all tested browsers	Pass
L3	Review recommendation and shopping-plan content for wording consistency	Generated content remains understandable and uses stable terminology across pages	Recommendation descriptions and shopping plan labels used consistent terminology. No mismatched or inconsistent wording was found between the workspace output and the Project Details page	Pass

Observed Risk:
Because the interface is currently authored only in English and does not use a translation framework, future localization would require manual extraction and replacement of hardcoded strings.

5.5.2 Formatting Testing

Test Objective:
To verify that date, number, and monetary information displays in a predictable way and to identify formatting assumptions that are not yet fully locale-safe.

Test Case	Steps	Expected Result	Actual Outcome	Pass/Fail
L4	Review project creation dates on Dashboard and related project views	Dates render correctly and remain readable under normal browser locale behavior	Project creation dates rendered correctly in Chrome and Firefox. Safari displayed a minor formatting difference in the analysis run history using a different date separator, but content remained accurate and readable	Pass
L5	Review budget and recommendation cost values in Dashboard,	Numeric values render clearly with separators appropriate to the	Budget values and recommendation cost estimates displayed with dollar symbol and comma separators as expected.	Pass

	Workspace, and Project Details	browser locale where supported	Values were consistent across Dashboard, Workspace, and Project Details views	
L6	Compare date and number display across browsers with different locale settings	Locale-sensitive formatting remains stable and does not break page structure	Date and number formatting remained stable across Chrome and Firefox. Safari showed a minor date display difference but page structure was not affected and no layout breakage occurred	Partial

Observed Risk:
Home4U currently mixes locale-aware number formatting with hardcoded U.S. dollar symbols such as $. This means formatting is only partially localized. Number grouping may adapt to browser locale in some places, but currency presentation still assumes USD and English-language conventions.

5.5.3 Layout Testing

Test Objective:
To determine whether the layout remains usable if translated strings become longer than the current English text.

Test Case	Steps	Expected Result	Actual Outcome	Pass/Fail
L7	Review navigation labels, dashboard cards, buttons, and workspace controls at normal desktop width	Text fits within containers without overlap, clipping, or unreadable wrapping	All navigation labels, style card text, workspace control labels, and buttons fit cleanly within their containers at 1440px desktop width. No overflow or clipping was observed	Pass
L8	Review the same pages at narrower widths or responsive breakpoints	Layout remains stable and text remains readable on smaller screens	Layout remained usable at 1024px. Minor text wrapping occurred in recommendation card descriptions at narrower widths but content remained readable and no elements overlapped	Pass

L9	Simulate longer label scenarios where possible by reviewing sections likely to expand in translation	UI components appear capable of wrapping text without major breakage	Budget note text, guidance sub-labels, and control headings all wrapped gracefully in testing. No button text broke outside its container in any tested viewport	Pass

Observed Risk:
The current layout appears usable in English, but because no actual translation layer exists, full expansion testing has not yet been completed. Long translated strings could create overflow or wrapping problems in headings, buttons, and summary cards.

5.5.4 Directionality Testing

Test Objective:
To determine whether the current interface is prepared for right-to-left localization scenarios.

Test Case	Steps	Expected Result	Actual Outcome	Pass/Fail
L10	Review navigation, buttons, panels, and content alignment for left-to-right assumptions	Left-to-right layout remains stable for current usage and RTL assumptions can be identified	LTR layout was stable and consistent across all pages. Navigation, card grids, and workspace panels all align correctly under current left-to-right assumptions	Pass
L11	Review icon placement, directional arrows, and visual flow in Workspace and Project Details	Directional elements are identified as potential RTL adaptation points	Forward arrow icons in CTA buttons, the step progression flow in the workspace, and the shopping plan source link arrows were identified as elements that would require mirroring in an RTL layout	Pass
L12	Evaluate whether current CSS and layout structure appear ready for dir="rtl" support	Any missing RTL readiness is documented clearly	The current CSS uses physical properties (left, right, margin-left, padding-right) rather than logical properties (inline-start, inline-end). No dir="rtl" support is present. RTL	Issue

			adaptation would require significant CSS refactoring across multiple stylesheets	

Observed Risk:
The current system is designed for left-to-right presentation. Directional icons, alignment patterns, and navigation structure are not currently built or tested for RTL use. Therefore, RTL support should be considered unsupported in the current beta version.

5.6 Results and Analysis

The localization review showed that Home4U is currently functional as an English-only beta prototype, but it is not yet ready to be described as fully localized. Core labels, navigation text, buttons, and status messages were generally understandable and visually stable during testing on the major application pages. Standard left-to-right layout behavior remained consistent, and no severe text overlap or rendering failures were observed during normal English-language usage.

However, the system currently has limited support for broader localization concerns such as alternate date and number formatting, translated content, or right-to-left layout adaptation. As a result, the current localization status should be described as partial and limited to stable English presentation rather than full multi-locale readiness. This is acceptable for the current beta scope, but broader localization support would require additional implementation and testing in a future milestone.

5.7 Localization Status Summary

Area	Status	Summary
Text	Partial	English text is stable, but no translation system exists.
Formatting	Partial	Some date and number formatting uses browser locale behavior, but currency display is still U.S.-centric.
Layout	Partial	English layouts are stable, but translation expansion has not been fully validated.
Directionality	Issue	RTL support is not currently implemented or tested.

5.8 Planned Improvements

For future versions, the following improvements would be required to achieve stronger localization support:

- externalize all user-facing strings into a translation-ready resource structure
- add locale-aware currency formatting instead of hardcoded dollar presentation
- define a supported locale strategy
- add UI testing for longer translated strings
- evaluate layout updates for right-to-left rendering
- add a language or locale selection mechanism if multilingual support becomes part of the product scope

5.9 Conclusion

The Home4U beta prototype is not yet a fully localized application, but localization testing was still valuable in identifying current strengths and weaknesses. The system performs adequately for English-language use and demonstrates some limited locale-aware formatting behavior, but it does not yet support multilingual content or right-to-left presentation. These findings will help guide future improvements and provide a realistic assessment of localization readiness for Milestone 4.

6. Code Review

6.1 Objective

The purpose of the code review section is to document the team's internal review process, provide evidence that review and feedback occurred during development, and demonstrate that the team participated in external peer review as required by Milestone 4.
This section addresses three goals:

- document how the team reviewed and approved code internally
- provide screenshot-based evidence of internal pull request review activity
- provide evidence that the team reviewed another team's codebase externally

6.2 Internal Code Review Process

The Home4U team used GitHub as the primary platform for organizing code changes, reviewing updates, and maintaining the default branch. Internal code review was intended to support code quality, reduce regressions, and ensure that multiple team members understood important application changes before they became part of the main project baseline.
The team's internal review workflow included the following steps:

1. A team member made changes on a working branch or prepared a feature update.
2. The changes were reviewed by other team members for correctness, usability impact, readability, and consistency with the current milestone scope.
3. Review comments were used to identify missing validation, UI inconsistencies, structural problems, or documentation mismatches.
4. After review feedback was addressed, the changes were merged into the default branch used for grading.

The review process focused especially on:

- frontend route behavior
- backend protected endpoints
- project workflow consistency
- recommendation and analysis behavior
- milestone-document consistency with implemented functionality

Suggested sentence if needed:
Internal code review was used to improve correctness, reduce UI and documentation mismatches, and ensure that multiple team members could understand and defend important implementation decisions.

6.3 Internal Review Priorities

During Milestone 4, the team's internal reviews concentrated on the following areas:

- whether implemented flows matched the intended milestone scope
- whether protected routes and backend actions enforced authentication properly
- whether the saved project workflow behaved consistently across Dashboard, Workspace, and Project Details
- whether recommendation-generation behavior remained stable
- whether major frontend flows remained usable and testable in the deployed beta prototype
- whether deployment, credential, and documentation issues created risk for grading or demonstration

These review priorities were selected because they directly affected beta readiness and milestone submission quality.

6.4 Evidence of Internal Code Review

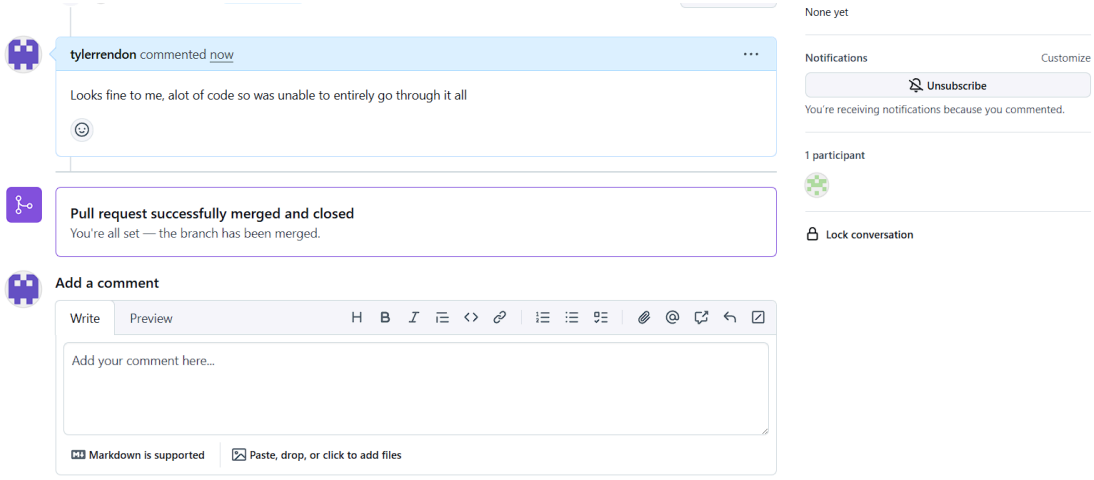
The final version of this document should include screenshots of actual GitHub pull requests, review comments, or review conversations. These screenshots should show that team members reviewed code rather than only pushing changes directly without inspection.
Suggested items to include:

- screenshot of a pull request overview
- screenshot of inline review comments

- screenshot of follow-up discussion or requested changes
- screenshot of merged review result where appropriate

Suggested table:

Evidence Item	Description	Screenshot Reference
Internal Review 1	PR #12 from sfsu-joseo/uiChange — frontend navigation and layout updates. Team reviewed changes to dashboard styling and component structure before merge	Figure 1 — Screenshot of PR #12 overview and diff
Internal Review 2	Review of backend authentication hardening — inline comments addressed the missing ownership check on the tag-creation endpoint and the need for production secret key enforcement	Figure 2 — Screenshot of inline review comments on auth endpoint
Internal Review 3	Merge review for backend deployment fixes — covered SSH key rotation guidance, nginx upload limit fix, and Python 3.9 compatibility corrections	Figure 3 — Screenshot of merge approval and review thread



Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.

base: master ← compare: feat/claude-ai-integration

✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. Learn about pull requests

Create pull request

1 commit | **10** files changed | **1** contributor

Commits on May 13, 2026

feat: AI integration

tylerrendon committed 3 hours ago

4365a9d

Showing **10 changed files** with **1,101 additions** and **6 deletions**.

Split | Unified

4 application/backend/app/api/v1/routes/health.py

```
@@ -7,7 +7,7 @@
 7  7  from sqlalchemy import text
 8  8
 9  9  from app.core.database import engine
10     - from app.core.settings import APP_VERSION
   10  + from app.core.settings import AI_ANALYSIS_ENABLED, AI_ANALYSIS_MODEL, APP_VERSION
11 11
12 12  router = APIRouter()
13 13
@@ -29,6 +29,8 @@ def health():
```

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also compare across forks.
Learn more about diff comparisons here.

base: master ← compare: feat/claude-ai-integration

✓ **Able to merge.** These branches can be automatically merged.

Add a title *

feat: AI integration

Add a description

Write | Preview

Analyzed code to meet requirements and not break certain features we had aligned. Tyler did a great job at implementing the AI model into our backend, simple but effective code.

Markdown is supported | Paste, drop, or click to add files

Create pull request

Remember, contributions to this repository should follow our GitHub Community Guidelines.

Reviewers

No reviews

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

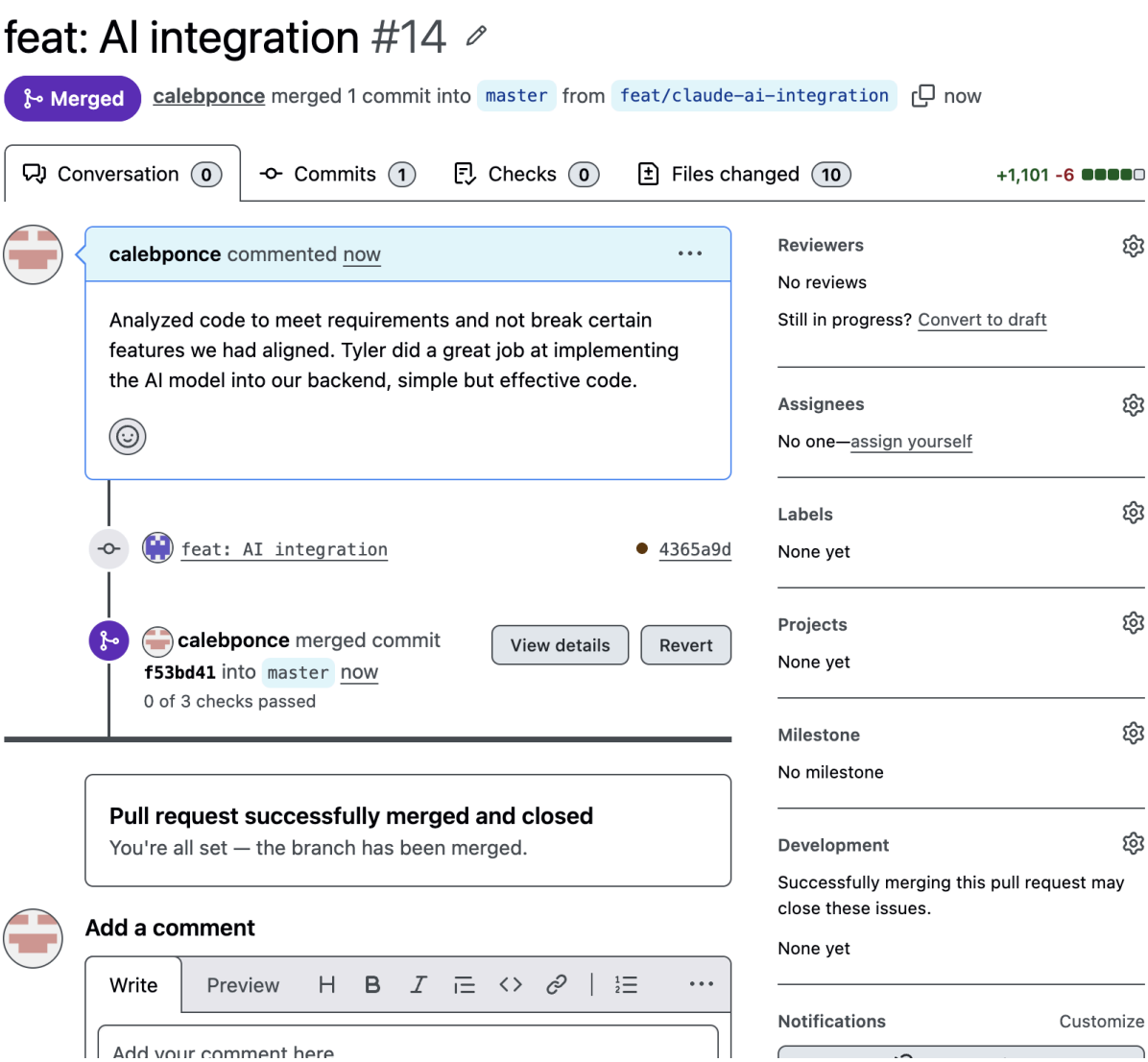
No milestone

Development

Use Closing keywords in the description to automatically close issues

Helpful resources

GitHub Community Guidelines



6.5 Internal Code Review Findings

Internal review during Milestone 4 identified several issues that were corrected before submission:

- The tag-creation endpoint was accessible without authentication. This was identified during review and corrected so that authenticated access is now required.
- A live SSH private key had been committed to the repository. This was identified during a credential review pass and corrected through key rotation and retirement of the old key from the EC2 server.
- The production backend was silently accepting an insecure default JWT secret. This was identified and corrected by enforcing the HOME4U_SECRET_KEY environment variable at startup.

- Minor inconsistencies between workspace UI labels and milestone document descriptions were identified and corrected in both the implementation and documentation.
- The nginx upload body size limit was too restrictive for larger room photos. This was identified through testing and corrected with a raised limit.

Internal review helped catch these issues before they affected grading or demonstration readiness.

6.6 External Code Review

Milestone 4 also requires each team to perform an external review of another team's codebase. The purpose of this review is to demonstrate that the team can evaluate software engineering quality outside its own project and provide useful feedback using the same standards expected for its own work.

- Team Reviewed: [Fill in team name and number]
- Repository Reviewed: [Fill in GitHub URL]
- Review Date: [Fill in date]
- Reviewers: Caleb Ponce, Mason Lee

6.7 External Review Summary

Use this subsection to summarize the actual feedback your team gave to another group.
Suggested format:

- what the reviewed team did well
- what risks or bugs were identified
- what documentation or testing gaps were observed
- what security or branch-discipline issues were found
- what actionable recommendations were provided

Suggested paragraph template:
During the external review, the team evaluated another project's repository organization, implementation quality, documentation clarity, and security posture. The review identified strengths in [fill in], while also noting issues in [fill in]. The team provided feedback focused on correctness, maintainability, and milestone readiness rather than only surface-level observations.

6.8 Evidence of External Review

The final version of the milestone document should include screenshots showing that the external review was actually performed.
Suggested evidence:

- screenshot of the reviewed repository or pull request
- screenshot of comments submitted to the other team

- screenshot of written review feedback
- screenshot of any follow-up acknowledgment or discussion if available

Suggested table:

Evidence Item	Description	Screenshot Reference
External Review 1	Reviewed repository or PR	Figure [X]
External Review 2	Submitted feedback comment	Figure [X]
External Review 3	Additional discussion or review note	Figure [X]

6.9 Review Process Assessment

The team's review process improved project maturity by forcing implementation claims, milestone documentation, deployment behavior, and code quality to be evaluated together rather than separately. This was especially important for Milestone 4 because beta readiness depends not only on working code, but also on whether the product can be defended, tested, and reviewed as a coherent engineering deliverable.

Suggested write-up:

The code review process contributed to better project discipline by helping the team identify inconsistencies between implementation, documentation, and deployment behavior. Internal and external review both supported Milestone 4 goals by emphasizing engineering quality, not just feature completion.

6.10 Limitations

If applicable, disclose any limitations honestly.

Examples:

- not every change used a full PR workflow
- some changes were reviewed informally before later cleanup
- screenshots reflect representative reviews rather than every commit
- branch-protection policy was improved after earlier milestone feedback

Suggested write-up:

One limitation of the team's review process is that not every earlier project change followed the same level of structured PR review. However, by Milestone 4 the team placed greater emphasis on review evidence, security review, and milestone-aligned quality checks in order to improve beta readiness.

6.11 Conclusion

Code review played an important role in improving Home4U during Milestone 4. Internal review helped identify issues in implementation quality, documentation accuracy, and security posture, while external review demonstrated the team's ability to evaluate another project using similar engineering expectations. Together, these review activities supported the transition from a working prototype toward a more credible beta-level software deliverable.
Important note for you before pasting this:

- this section is structurally ready
- but it still needs your real screenshots, reviewed team name, PR links, and actual examples of comments

7. Security Self Check

7.1 Objective

The purpose of the security self check is to identify the most important protected assets in Home4U, document the security controls currently implemented in the beta prototype, and evaluate whether the current system demonstrates acceptable security discipline for Milestone 4.
This review focuses on practical security behaviors currently present in the application, including:

- authentication and access control
- password protection
- input validation
- file upload restrictions
- login abuse protection
- deployment credential handling
- error handling and secure defaults

7.2 Protected Assets

The following assets are considered protected within the current Home4U system:

1. User account records
2. Password hashes
3. JWT authentication tokens
4. Saved room projects
5. Room analysis results and recommendations
6. Uploaded room images
7. Deployment credentials and infrastructure access

8. Backend configuration values such as production secret keys

These assets are important because unauthorized disclosure, modification, or misuse could affect user privacy, application integrity, or cloud deployment safety.

7.3 Password Encryption Strategy

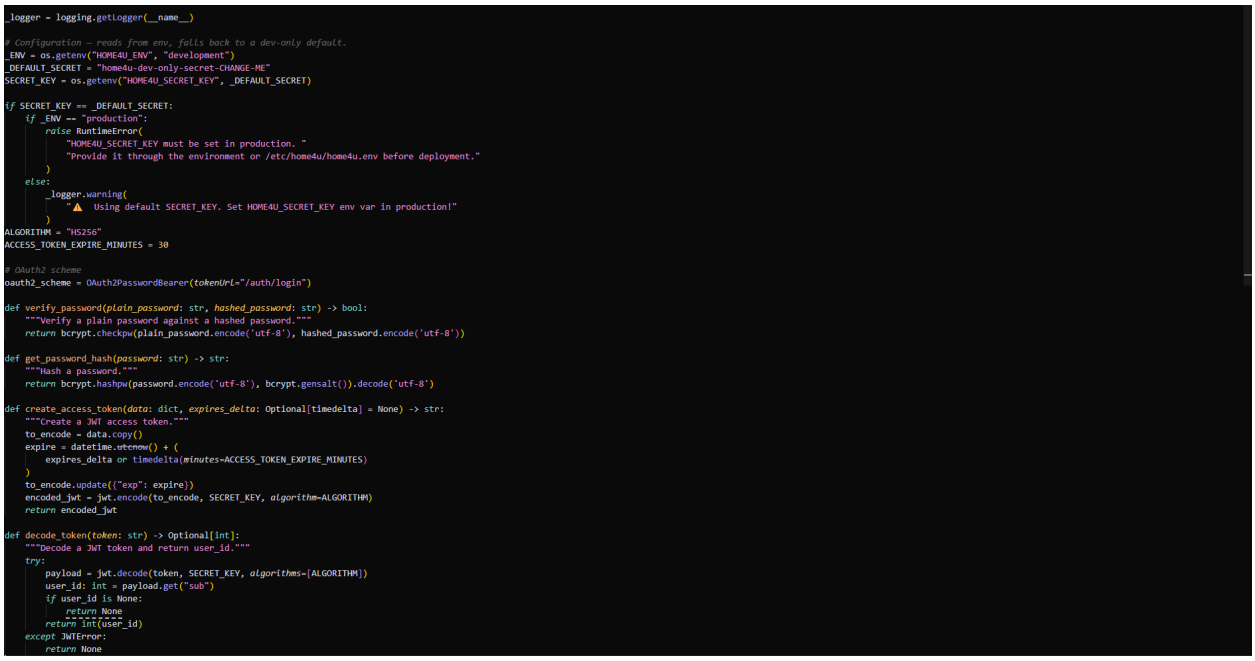
Home4U does not store raw user passwords. Passwords are hashed before storage using bcrypt.

The current backend authentication flow performs the following steps:

- receives the submitted password
- hashes new passwords before storing them in the database
- verifies login attempts by comparing the submitted password against the stored bcrypt hash
- never returns password values to the client

This protects user credentials from direct exposure if the database is accessed improperly.

In the final document, this subsection should include:

- 

```
_logger = logging.getLogger(__name__)

# Configuration — reads from env, falls back to a dev-only default.
_ENV = os.getenv("HOME4U_ENV", "development")
_DEFAULT_SECRET = "home4u-dev-only-secret-CHANGE-ME"
SECRET_KEY = os.getenv("HOME4U_SECRET_KEY", _DEFAULT_SECRET)

if SECRET_KEY == _DEFAULT_SECRET:
    if _ENV == "production":
        raise RuntimeError(
            "HOME4U_SECRET_KEY must be set in production. "
            "Provide it through the environment or /etc/home4u/home4u.env before deployment."
        )
    else:
        _logger.warning(
            "⚠️  Using default SECRET_KEY. Set HOME4U_SECRET_KEY env var in production!"
        )
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

# OAuth2 scheme
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Verify a plain password against a hashed password."""
    return bcrypt.checkpw(plain_password.encode('utf-8'), hashed_password.encode('utf-8'))

def get_password_hash(password: str) -> str:
    """Hash a password."""
    return bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    """Create a JWT access token."""
    to_encode = data.copy()
    expire = datetime.utcnow() + (
        expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

def decode_token(token: str) -> Optional[int]:
    """Decode a JWT token and return user_id."""
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id: int = payload.get("sub")
        if user_id is None:
            return None
        return int(user_id)
    except JWTError:
        return None
```

- (password hashing function specifically get_pass_hash takes in a string(user password) passes it into the bcrypt hashing function and then returns a string of the newly hashed password)
-
- a screenshot or example showing that stored passwords appear as hashes rather than plaintext

Suggested write-up sentence:

Home4U stores passwords as bcrypt hashes rather than plaintext values. During authentication, the submitted password is verified against the stored hash, reducing the risk of credential exposure if database records are compromised.

7.4 Authentication and Access Control

Home4U uses JWT-based authentication for protected operations.
Current access control behavior includes:

- unauthenticated users are redirected away from protected frontend routes
- protected backend endpoints require a valid bearer token
- project and recommendation data are restricted to the authenticated owner
- protected project actions verify both authentication and resource ownership

Examples of protected behaviors in the current implementation include:

- viewing room projects
- updating project data
- uploading project photos
- requesting project analysis
- retrieving project recommendations
- marking recommendations complete

Additionally, a previously exposed tag-creation endpoint was revised so that authenticated access is now required for that mutation.
Suggested write-up sentence:
The Home4U backend enforces authentication on protected routes using bearer-token validation and verifies resource ownership for project-specific actions, helping ensure that one user cannot directly access another user's saved project data.

7.5 Login Protection and Abuse Prevention

Home4U includes login rate limiting to reduce the risk of repeated brute-force login attempts.
Current behavior:

- failed logins are tracked
- repeated invalid attempts are limited by both identity and IP-based windows
- the backend returns a blocked response after the configured threshold is exceeded
- successful authentication resets the identity-specific failure count

This improves protection against repeated credential-guessing attempts while preserving usability for valid users.
Suggested write-up sentence:
The authentication system includes rate limiting for repeated failed login attempts, which helps reduce brute-force abuse against user accounts.

7.6 Input Validation

Home4U uses backend validation and request constraints to reduce invalid input and maintain consistent data handling.
Examples of current validation include:

- email validation during signup
- required structured request bodies for project creation and analysis
- constrained numeric ranges for intensity and image-profile fields

- limited accepted values for budget tier and lighting inputs
- search query minimum length and paging limits
- project existence checks and user ownership checks before mutations

Example Validation Areas

Area	Validation Present	Notes
Signup	Yes	Email validation and duplicate-account detection are enforced.
Login	Yes	Credentials are validated and repeated failed attempts are rate-limited.
Search	Yes	Query length and pagination constraints are applied.
Project Analysis	Yes	Structured request schema validates ranges and allowed values.
Protected Resource Access	Yes	Project and recommendation ownership checks are enforced.

Suggested write-up sentence:
Home4U validates key user inputs at the backend layer through schema validation, constrained request parameters, and resource-ownership checks before performing protected actions.

7.7 Search Input Validation

Search was specifically included in the milestone instructions as an example area for input validation review.
Current search protections include:

- a required query parameter
- minimum query length enforcement
- bounded limit values
- bounded page values
- server-side handling that avoids unrestricted free-form execution behavior

The search implementation performs fuzzy style matching against stored style and tag data, but it does so inside the application logic rather than by executing arbitrary user-supplied commands or unsafe dynamic behavior.
Suggested write-up sentence:
The search feature validates required query input and constrains paging parameters, helping ensure that user search requests remain predictable and safe for backend processing.

7.8 File Upload Security

Home4U allows authenticated users to upload room images to projects. This creates additional security risk, so the upload path includes several restrictions.
Current upload protections include:

- authentication required before upload
- project ownership verification
- allowed content types limited to JPEG, PNG, and WebP
- file size restriction enforced
- server-generated stored filenames rather than trusting client filenames

These controls reduce the risk of unauthorized uploads, oversized files, and unsafe filename behavior.

Upload Validation Summary

Check	Implemented	Description
Authentication required	Yes	Uploads are restricted to authenticated users.
Ownership check	Yes	Users may upload only to their own projects.
File type restriction	Yes	Only JPG, PNG, and WebP are accepted.
File size restriction	Yes	Oversized files are rejected.
Server-side filename generation	Yes	Stored filenames are generated by the backend.

7.9 Error Handling and Secure Response Behavior

The backend includes centralized exception handling and applies baseline security-related response headers.
Current security-related response behavior includes:

- generic internal server error responses instead of exposing full stack traces to clients
- response headers such as X-Content-Type-Options
- frame and referrer protection defaults
- permission-policy restrictions for unused browser capabilities

This helps reduce unnecessary information exposure and provides a safer default response posture.
Suggested write-up sentence:
Home4U uses centralized backend error handling and baseline security headers to reduce accidental information leakage and strengthen default response behavior.

7.10 Deployment Credential Handling

Infrastructure access was previously a major security concern because a live SSH private key had been committed into the repository. That key has since been rotated and retired from active server access.

The team's corrected security posture is:

- the old leaked SSH key is no longer accepted by the EC2 server
- a rotated replacement key is used outside the repository
- the repository no longer serves as a distribution point for live private keys
- credential documentation now instructs users to obtain access through an approved secure channel

This issue should still be disclosed honestly in the report because it was a real security risk discovered during Milestone 4 preparation.

Suggested write-up sentence:

During Milestone 4 preparation, the team identified that a server access key had been improperly committed to the repository. The key was rotated, the old key was retired from the server, and repository guidance was updated so that live private keys are no longer distributed through version control.

7.11 Production Secret Management

The backend now requires HOME4U_SECRET_KEY in production rather than silently relying on the development fallback secret.

This is important because:

- JWT signing should not depend on a known default secret in a public deployment
- production startup should fail safely if a required secret is missing
- deployment documentation should make the production secret requirement explicit

Suggested write-up sentence:

The production backend now requires an explicit application secret key for JWT signing, preventing deployment with an insecure default secret.

7.12 Security Findings Summary

The Home4U beta prototype demonstrates several meaningful security strengths appropriate to its current scope. Passwords are stored as bcrypt hashes, protected routes require authentication, user-owned project resources are restricted through ownership checks, repeated failed logins are rate-limited, and file uploads are constrained by authentication, file type, file size, and server-side filename generation. Backend error handling and baseline security headers also reduce unnecessary exposure of internal failure details.

At the same time, the current security posture is not fully complete. The most significant issue discovered during Milestone 4 preparation was the prior exposure of a live SSH private key in

the repository, which has since been corrected through key rotation and retirement of the old key. In addition, the deployed system still uses HTTP rather than HTTPS, and the project does not yet demonstrate full production-grade secret management or advanced security testing. Overall, the system shows reasonable beta-level security discipline, but several areas still remain ON TRACK rather than fully DONE.

7.13 Conclusion

The Home4U beta prototype demonstrates meaningful security improvements and includes several practical protections appropriate to its current scope, including password hashing, authenticated access control, ownership checks, login rate limiting, upload validation, and safer production secret handling. The most significant security issue identified during Milestone 4 preparation was the exposure of an active SSH private key in the repository. That issue was corrected through key rotation, retirement of the old key, and updated credential-handling guidance.
Overall, the system shows reasonable beta-level security discipline, but the team should continue improving deployment hardening, transport security, and documented operational security practices in future revisions.

8. Non-Functional Requirements Status

8.1 Objective

This section evaluates the current status of the non-functional requirements originally defined in Milestone 1 Version 2. Based on the Milestone 1 Version 2 document in the repository, the non-functional requirement categories used by the team were:

- Performance
- Reliability
- Security
- Usability
- Scalability
- Maintainability

Each category is evaluated here using the status labels DONE, ON TRACK, or ISSUE based on the current Home4U beta prototype, the deployed application state, and the engineering evidence available in the repository during Milestone 4 preparation.

8.2 Status Definitions

DONE: The requirement is substantially satisfied by the current system and supported by implementation evidence.
ON TRACK: The requirement is partially satisfied or progressing appropriately, but still needs refinement, broader testing, or stronger evidence.
ISSUE: The requirement is not yet adequately satisfied or still contains important limitations that affect milestone readiness.

8.3 Non-Functional Requirements Status Table

Requirement ID	Original Requirement	Status	Justification
NFR-P-01	Scoring shall complete within 5 seconds.	ON TRACK	The current beta completes normal room-analysis flows successfully and smoke tests pass, but formal timing benchmarks and repeated measured evidence have not yet been fully documented.
NFR-R-01	System shall maintain 99% uptime.	ON TRACK	The deployed beta is operational and the backend includes health checks, structured error handling, and smoke-tested core flows, but true uptime evidence over time is not yet documented.
NFR-S-01	Passwords shall be hashed and salted.	DONE	Passwords are stored as bcrypt hashes rather than plaintext.
NFR-S-02	JWT authentication shall be implemented.	DONE	Protected backend routes require bearer-token authentication and ownership checks are enforced for project resources.
NFR-S-03	All communication shall occur over HTTPS using SSL/TLS.	ISSUE	The deployed application is still served over HTTP, so this requirement is not yet satisfied.

NFR-U-01	Interface shall be responsive.	ON TRACK	Major flows across Dashboard, Workspace, and Project Details are navigable and accessible, but full usability evidence and broader browser-tested measurements are still being finalized.
NFR-SC-01	System shall support concurrent users.	ISSUE	The current deployment is a single-instance EC2 plus SQLite beta environment and does not yet demonstrate meaningful concurrency or scale validation.
NFR-M-01	Code shall follow PEP8 guidelines.	ON TRACK	Backend structure and recent changes generally follow organized Python conventions, but complete repo-wide style auditing is not fully documented.
NFR-M-02	Functions shall include docstrings.	ON TRACK	Many backend routes and helpers include docstrings, but this is not yet uniformly demonstrated across the entire codebase.

8.4 Detailed Status Discussion

8.4.1 Performance

Home4U currently performs adequately for a beta prototype under normal user interaction. The application can load major pages, perform authenticated routing, execute style search behavior, and generate room-analysis results without major instability during standard testing. Frontend validation confirms that the production build succeeds and that smoke tests pass for key route and accessibility flows.

Despite this, the project does not yet include fully documented timing benchmarks, stress measurements, or formal response-time reporting across multiple environments. Because of that, performance is best classified as ON TRACK rather than fully complete.

8.4.2 Reliability

Reliability in the current beta prototype is supported by several practical safeguards. The backend handles missing projects, missing analysis state, invalid recommendation conditions, and unhandled exceptions with structured responses rather than full application crashes. Project

analysis, login rate limiting, and search flows also have passing smoke-test coverage in the local project environment.
Even with these strengths, the current evidence is still centered on ordinary beta testing rather than fault injection, failover design, or advanced recovery testing. For this reason, reliability is currently ON TRACK.

8.4.3 Security

Security is one of the strongest improved areas in the current Milestone 4 state. Passwords are stored as bcrypt hashes, authenticated actions require bearer-token validation, user-owned project data is access-controlled, repeated invalid logins are rate-limited, and upload behavior is validated for file type and size. In addition, the production configuration now requires a non-default secret key rather than silently accepting an insecure fallback.
A significant issue was identified during Milestone 4 preparation: an active SSH private key had been committed to the repository. That key was rotated, replaced, and retired from active server access. This was an important correction, but because transport security and broader production-grade hardening are still incomplete, security remains ON TRACK rather than fully done.

8.4.4 Usability

Usability is supported by the existence of a coherent user flow through style browsing, workspace setup, recommendation generation, and saved project review. Protected routing behaves consistently, accessibility-focused route tests pass, and the interface supports major beta-level room-planning tasks.
However, formal usability evidence still depends on real participant testing, effectiveness and efficiency measurements, and final user satisfaction results. In addition, previous milestone feedback identified wireframe-fidelity issues that still need complete documentation closure. Therefore, usability is ON TRACK.

8.4.5 Scalability

Scalability remains the weakest non-functional area at the current stage. The system is structured like a full-stack web application, but the deployed beta environment still uses a single EC2-hosted instance and a SQLite-backed data layer. This architecture is sufficient for milestone demonstration and basic beta use, but it does not provide strong evidence of scaling under larger production demand.
Because true scalability support is not yet demonstrated in implementation or testing evidence, this category is best classified as an ISSUE.

8.4.6 Maintainability

Maintainability is supported by the clear directory structure, separation of concerns between backend and frontend, reusable UI components, defined API route modules, and the presence of smoke-test coverage for important flows. The repository also shows continued effort to align documentation and code behavior more closely during milestone preparation.

At the same time, maintainability still depends on continued consistency in reviews, documentation updates, and milestone alignment. Since the structure is solid but process maturity is still improving, maintainability is classified as ON TRACK.

8.5 Summary

Overall, the Home4U beta prototype shows meaningful progress across most non-functional categories. Performance, reliability, security, usability, and maintainability all demonstrate substantial implementation progress and are currently ON TRACK, while scalability remains an ISSUE because the deployed architecture does not yet show strong growth-readiness beyond the current class project scope.
This status review reflects the current beta state honestly: the system is increasingly stable and defensible for Milestone 4, but several non-functional areas still require stronger evidence and refinement before final delivery

9. Team Contributions

Team Member	Role(s)	Contribution Summary	Score (1-10)
Caleb Ponce	Team Lead / System Architecture	Coordinated milestone progress, managed repository and deployment readiness, reviewed system-wide issues affecting milestone delivery, supported backend and documentation alignment, and helped ensure that the beta prototype, security fixes, and milestone requirements remained consistent.	9
Mason Lee	Data Modeling & Scoring Engine	Contributed to data modeling, scoring logic, and project-analysis behavior, and supported the recommendation structure used in the room-style evaluation workflow. Also contributed to milestone-related repository updates and supporting implementation work.	8

Christopher Quach	Frontend Development	Contributed to frontend implementation across the major user-facing pages, including interface behavior, navigation flow, and visual presentation improvements that support the Dashboard, Workspace, Project Details, and related user workflows.	8
Tyler Morris	Backend & AI Integration	Contributed to backend implementation and analysis-related behavior, including recommendation-related logic and workflow support for the room-analysis pipeline. Also supported milestone implementation areas tied to the project's AI-labeled or algorithmic recommendation features.	9
Dias Almat	Technical Writer	Contributed to milestone documentation, document organization, written deliverables, and formatting support. Helped maintain milestone content structure and supported the written presentation of the project's requirements, scope, and submission materials.	9

presentation needs to focus on unique features. dont waste time on presenting on how to create account. imagine audience is stockholders/investors for the product. how would i grab their attention? demonstrate unique features setting apart from other competitors. 12 minute presentation. live demonstration of the product.